

# Вопросы к экзамену МиСПИ 2026

## 1. ISO/IEC 12207:2010: Жизненный цикл ПО. Группы процессов ЖЦ.

стр. 7

Жизненный цикл – время существования программы от момента замысла, до вывода ее из эксплуатации. Все этапы ЖЦ описаны в ISO

Основные этапы:

- Разработка требований (формулирует заказчик)
- Анализ
- Проектирование
- Разработка и Тестирование
- Внедрение
- Эксплуатация (В процессе эксплуатации происходит поддержка пользователей)
- Вывод из эксплуатации

Группы процессов:

- Согласования (2)
- Орг. обоснования (5)
- Проектов (7)
- Тех. процессов (11)
- Реализация ПС (7)
- Поддержки ПС (8)
- Повторного использования ПС (3)

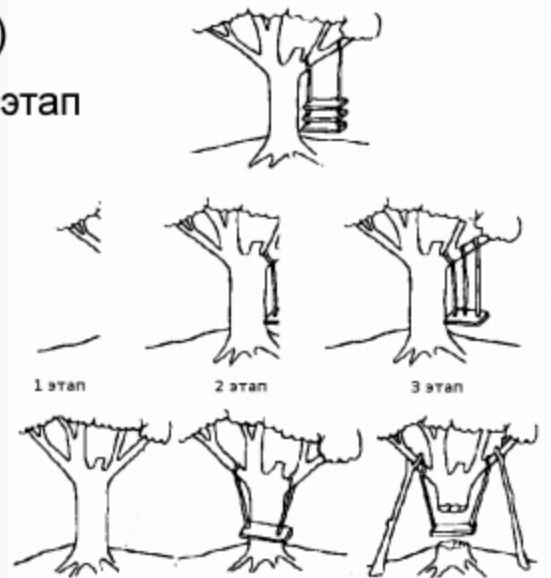
стр. 8

[wiki](#)

## 2. Модели ЖЦ.

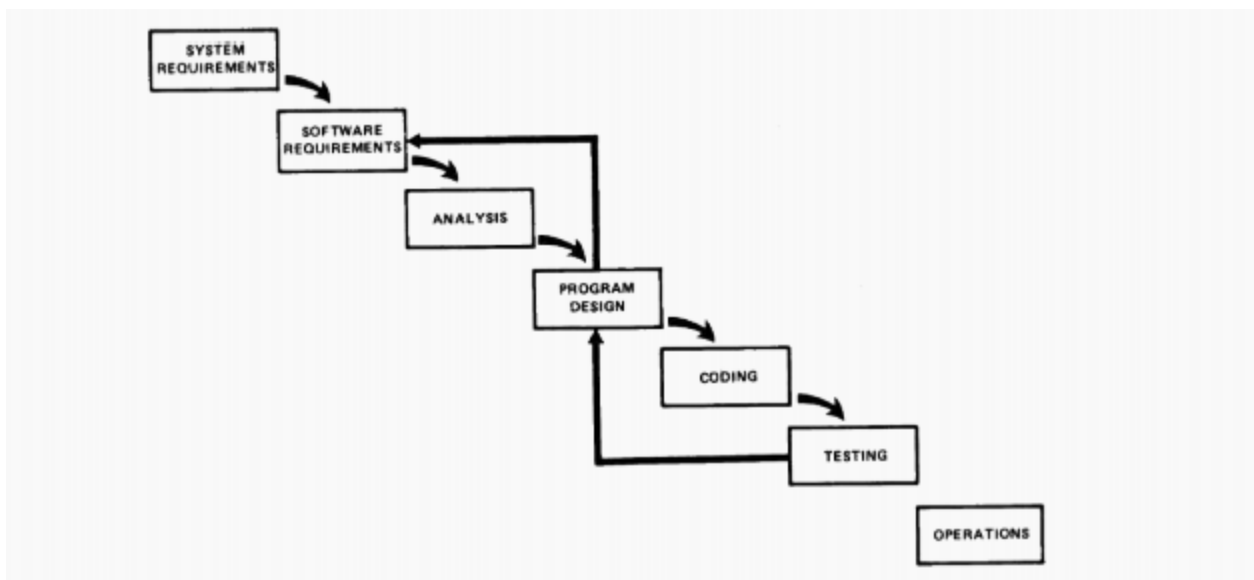
стр. 10

- Последовательная (однократная)
  - Определены все требования, один этап разработки
- Инкрементная
  - Определены все требования, несколько этапов
- Эволюционная
  - Определены не все требования, несколько этапов
- Формальных преобразований



### 3. Водопадная (каскадная) модель.

стр. 12



Разработана в 60-х. Критически описана Ройсом в 70-х.

Каскадная модель определяет последовательный переход на следующий этап после завершения предыдущего. Для этой модели характерна автоматизация отдельных несвязанных задач.

| Достоинства                                                                      | Недостатки                                                                                                                                  |
|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Хорошие показатели по срокам разработки и надежности при решении отдельных задач | Неприменимость к большим и сложным проектам из-за невозможности принять изменения требований к системе в течение длительного проектирования |
| Автоматизация отдельных несвязных задач                                          | Позднее тестирование                                                                                                                        |

## 4. Методология Ройса.

стр. 13

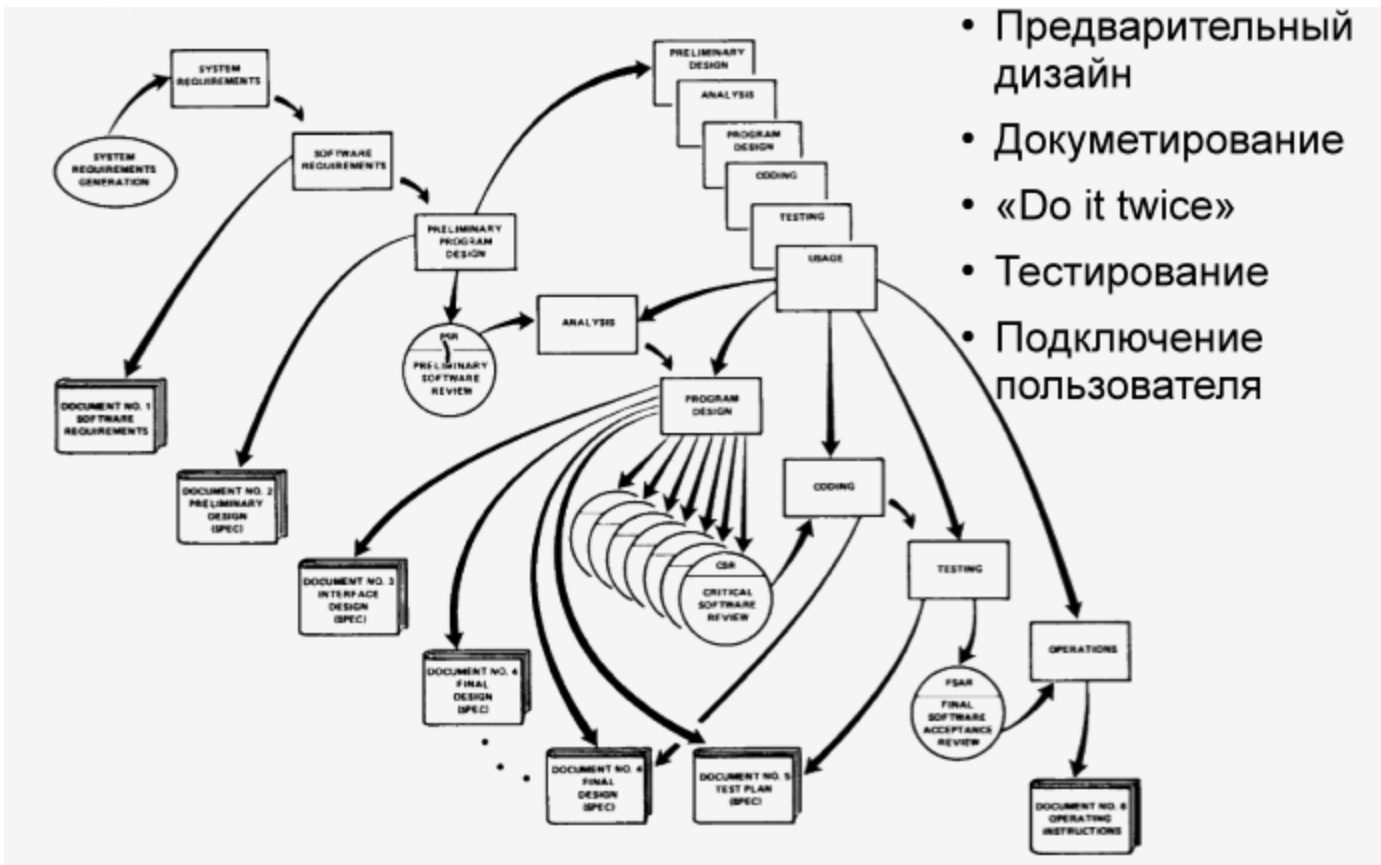
**Первый шаг:** 'Дизайн программы'. В нем дизайнеру предлагается спроектировать, определить и создать модели обработки данных и разработать документ: обзор будущей программы

**Второй шаг:** документирование дизайна, требования к системе, спецификация дизайна, план тестирования, инструкция по использованию.

**Третий шаг:** 'do it twice', тестовая разработка параллельно основному процессу и использование в качестве пилота для подтверждения или опровержения основных спецификаций ПО

**Четвертый шаг:** планирование контроль и мониторинг

**Пятый шаг:** подключения пользователя на ранних этапах



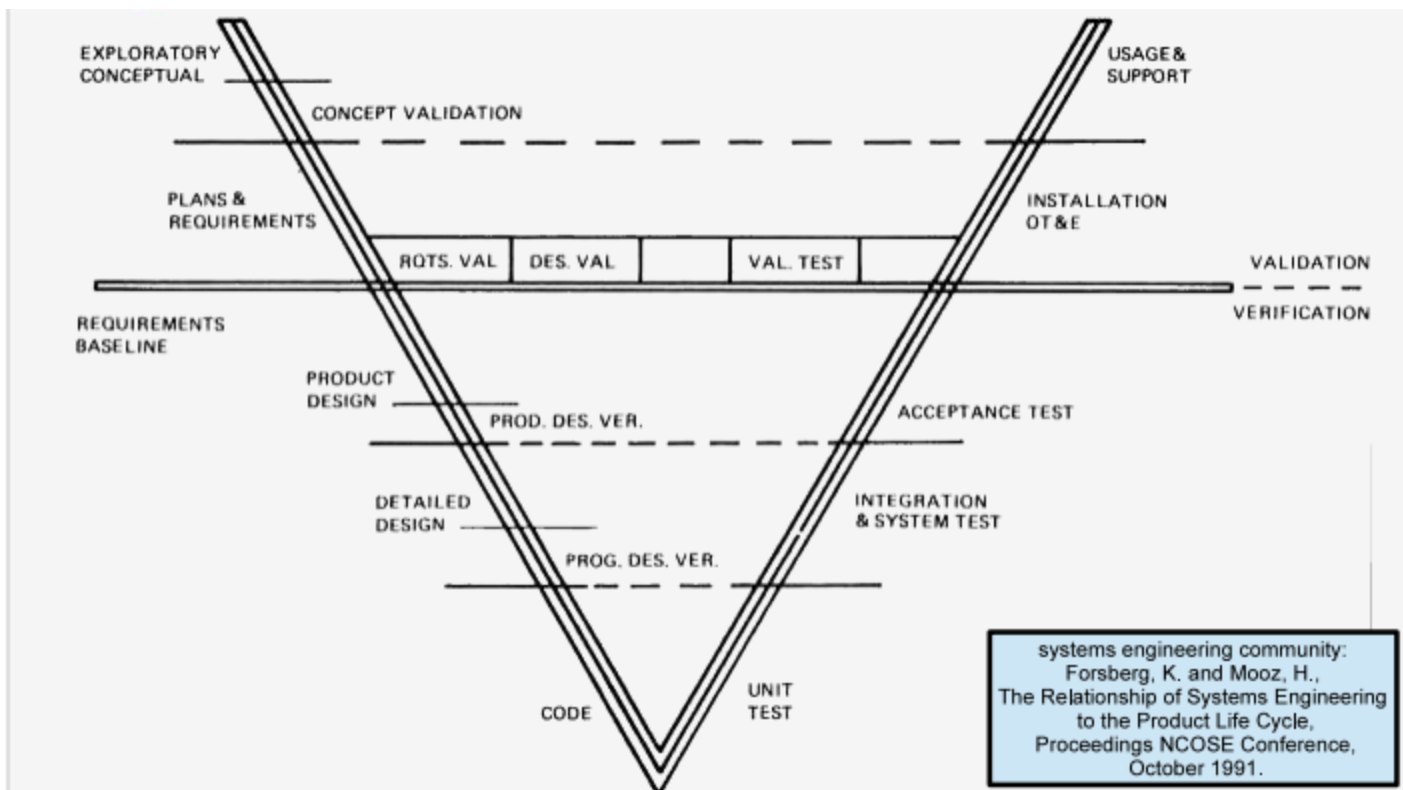
| Достоинства                                        | Недостатки |
|----------------------------------------------------|------------|
| Тестовая разработка параллельно основному процессу |            |
| Подключение пользователей                          |            |

## 5. Традиционная V-chart model J.Munson, B.Boehm.

стр. 14

Та же последовательность действий, что и в каскадной модели, но каждому уровню разработки свой уровень тестирования.

Модульное, интеграционное и системное тестирование проводятся последовательно на основании критериев верификации. Последним этапом является приемочное тестирование. Статическое тестирование может выполняться на ранней стадии разработки



| Достоинства                      | Недостатки |
|----------------------------------|------------|
| Тестирование ПО на каждой стадии |            |

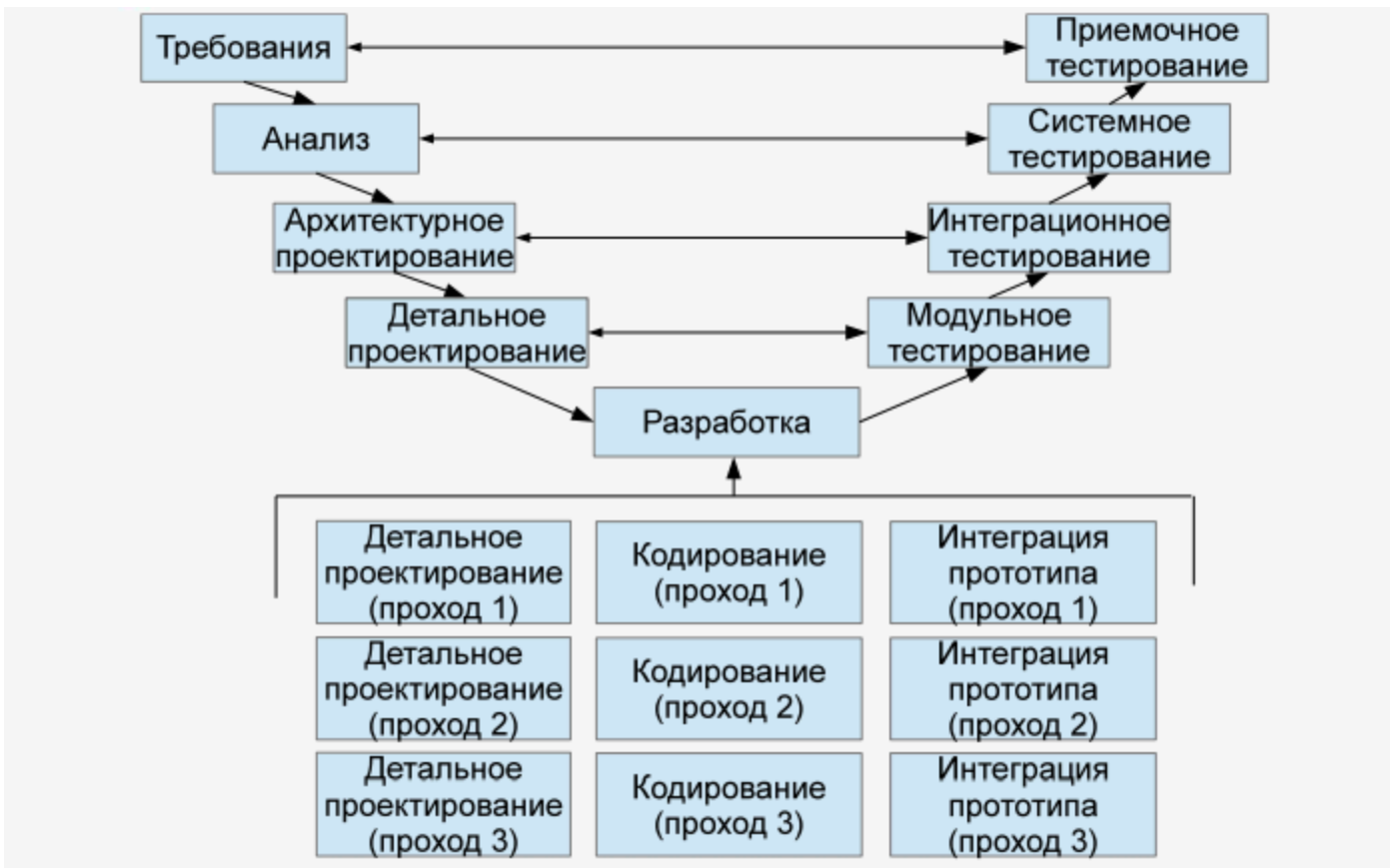
## 6. Многопроходная модель (Incremental model).

стр. 15

Цикл разделен на более мелкие легко создаваемые модули. Каждый модуль проходит через фазы определения требований, проектирования, кодирования, внедрения и тестирования.

Процедура разработки по инкрементной модели предполагает выпуск на первом большом этапе продукта в базовой функциональности, а затем уже последовательное добавление новых функций, так называемых «инкрементов».

Процесс продолжается до тех пор, пока не будет создана полная система. Заказчик может наблюдать за разработкой, вносить изменения, которые не сильно увеличат стоимость.



| Достоинства                                                                                                      | Недостатки                      |
|------------------------------------------------------------------------------------------------------------------|---------------------------------|
| Изменения заказчика не намного увеличивают стоимость разработки. Модель закладывает не сложную доработку системы | Устаревание архитектуры системы |

## 7. Модель прототипирования (80-е).

стр. 16

Модель позволяет создать прототип программного продукта до или в течение этапа составления требований к программному продукту.

Потенциальные пользователи работают с этим прототипом, определяя его сильные и слабые стороны, о результатах сообщают разработчикам программного продукта.

Если прототип не подходит, то разработка начинается заново.

Таким образом, обеспечивается обратная связь между пользователями и разработчиками, которая используется для изменения или корректировки спецификации требований к программному продукту



Жизненный цикл разработки программного продукта начинается с **разработки плана проекта**, затем выполняется **быстрый анализ**, после чего **создаются база данных, пользовательский интерфейс** и выполняется **разработка необходимых функций**.

В результате этой работы получается документ, содержащий частичную спецификацию требований к программному продукту. После этого определяются проблемы, над устранением которых совместно работают пользователи и разработчики. Этот процесс продолжается, пока пользователи не станут довольны.

| Достоинства                                                | Недостатки |
|------------------------------------------------------------|------------|
| Быстрая обратная связь                                     |            |
| Программный продукт точнее отвечает запросам пользователей |            |

## 8. RAD методология. 80-е

стр. 18

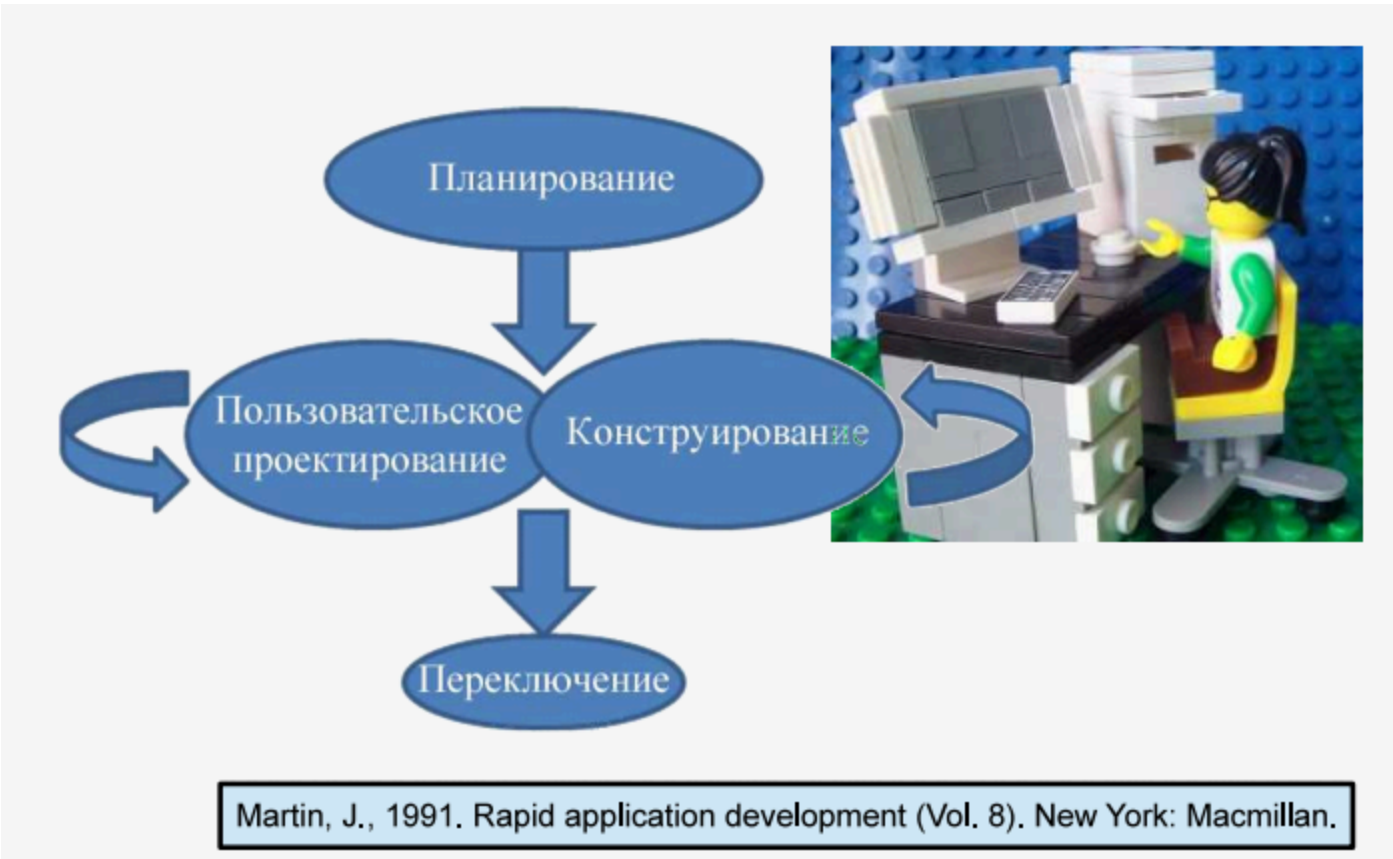
**Rapid Application Development** model или быстрая разработка приложений

Разновидность *инкрементной модели*. Пользователь принимает непосредственное участие в процессе разработки.

При помощи интерфейса, пользователь способен создавать простейшие функции.  
В RAD-модели компоненты или функции разрабатываются несколькими высококвалифицированными командами параллельно, будто несколько мини-проектов.

Временные рамки одного цикла жестко **ограничены**.

Созданные модули затем интегрируются в один рабочий прототип.  
Синергия позволяет очень быстро предоставить клиенту для обозрения что-то рабочее с целью получения обратной связи и внесения изменений.



Модель быстрой разработки приложений включает следующие фазы:

- Бизнес-моделирование
- Моделирование данных
- Моделирование процесса
- Сборка приложения
- Тестирование

| Достоинства                     | Недостатки |
|---------------------------------|------------|
| Обратная связь от пользователей |            |
|                                 |            |

## 9. Спиральная модель.

стр. 17

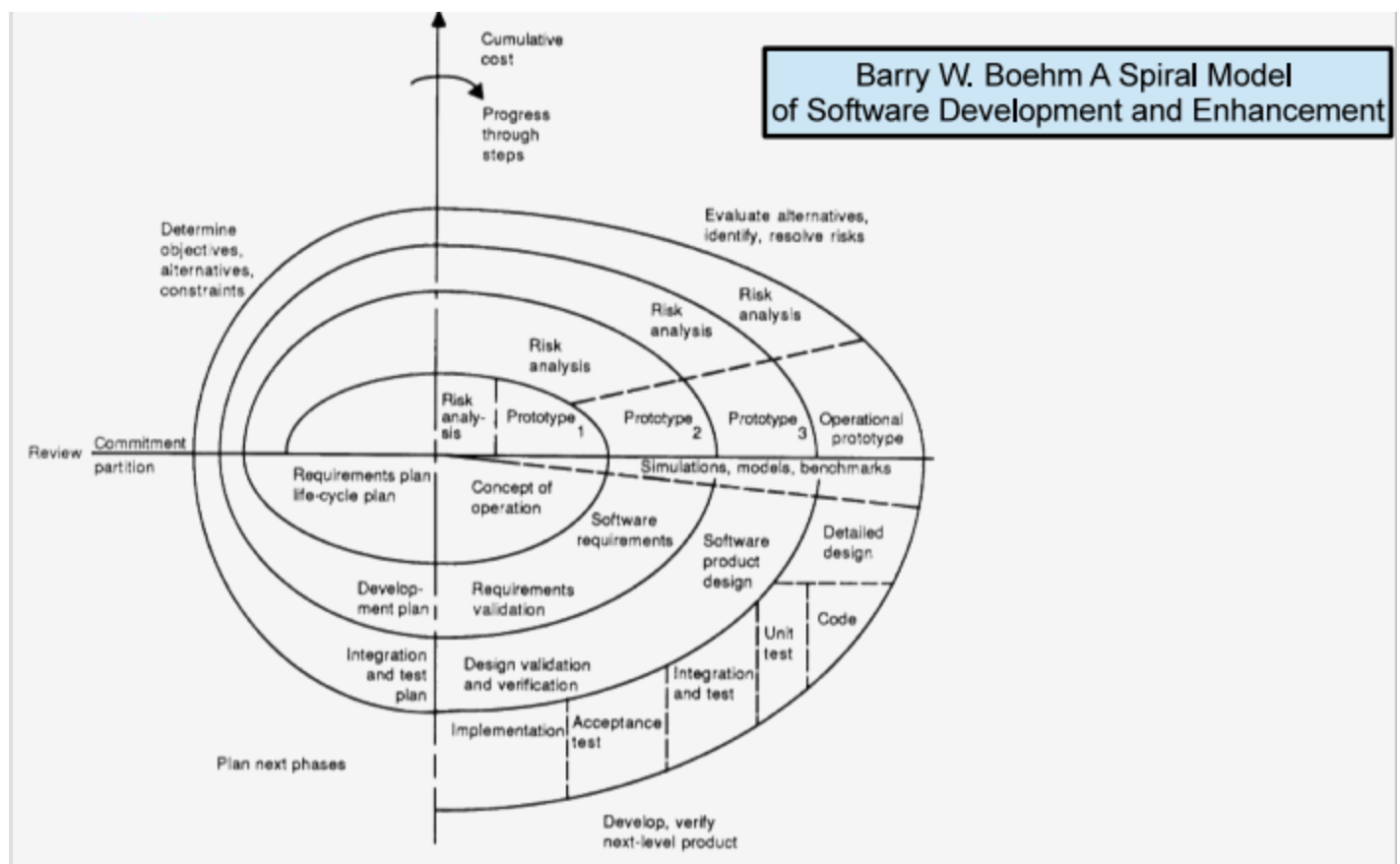
Спиральная модель = проектирование + поэтапное прототипирование

с целью сочетания преимуществ восходящей и нисходящей концепции.

Данный подход может оказаться довольно затратным в применении.

В спиральной модели особое внимание уделяется управлению рисками.

Контроль рисков, в свою очередь, требует проведения специфического анализа на каждой итерации. Изменения – неотъемлемая часть разработки.



| Достоинства                                  | Недостатки                           |
|----------------------------------------------|--------------------------------------|
| Изменения – неотъемлемая часть разработки    | Разработка может оказаться затратной |
| Особое внимание уделяется управлению рисками | Не подходит для маленьких проектов   |

# 10. UML Диаграммы: Структурные и поведенческие.

Основное назначение UML - графическое представление различных аспектов разработки ПО.



Рис. 1.6. Типы UML-диаграмм

стр. 34 [UML 2]

## Структурные диаграммы

Эти диаграммы используются для демонстрации статической структуры элементов в системе. Они могут изображать **архитектурную организацию системы, ее физические элементы, текущую конфигурацию, а также специфические элементы предметной области.**

## Поведенческие диаграммы

События, происходящие в системах программного обеспечения, являются динамическими:

- объекты создаются и уничтожаются,
- объекты передают сообщения другим объектам и системам,
- внешние события активизируют операции над определенными объектами.

# 11. UML: Use-case модель.

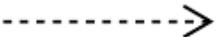
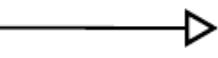
Это диаграмма динамического поведения в UML, которая моделирует функциональность системы с использованием участников, прецедентов и других важнейших объектов.

Актеры (actors) — это люди или организации, которые работают под определенными ролями внутри системы.

Отношение включения указывает на то, что поведение одного прецедента включается в некоторой точке в другой прецедент в качестве составного компонента.

Отношение расширения отражает возможное присоединение одного варианта использования к другому в некоторой точке.

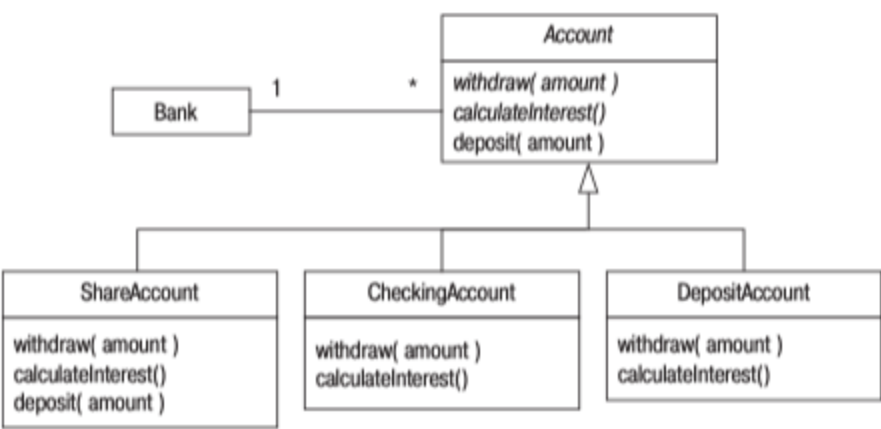
Таблица 1.1

| Тип<br>отношения | UML-синтаксис                                                                       |      | Краткая<br>семантика                                                                               | Раз-<br>дел |
|------------------|-------------------------------------------------------------------------------------|------|----------------------------------------------------------------------------------------------------|-------------|
|                  | источник                                                                            | цель |                                                                                                    |             |
| Зависимость      |  |      | Исходный элемент зависит от целевого элемента и изменение последнего может повлиять на первый.     | 9.5         |
| Ассоциация       |  |      | Описание набора связей между объектами.                                                            | 9.4         |
| Агрегация        |  |      | Целевой элемент является частью исходного элемента.                                                | 18.4        |
| Композиция       |  |      | Строгая (более ограниченная) форма агрегирования.                                                  | 18.5        |
| Включение        |  |      | Исходный элемент содержит целевой элемент.                                                         | 11.4        |
| Обобщение        |  |      | Исходный элемент является специализацией более обобщенного целевого элемента и может замещать его. | 10.2        |
| Реализация       |  |      | Исходный элемент гарантированно выполняет контракт, определенный целевым элементом.                | 12.3        |

# 12. UML: Диаграмма классов.

[Почитать](#)

Пример:



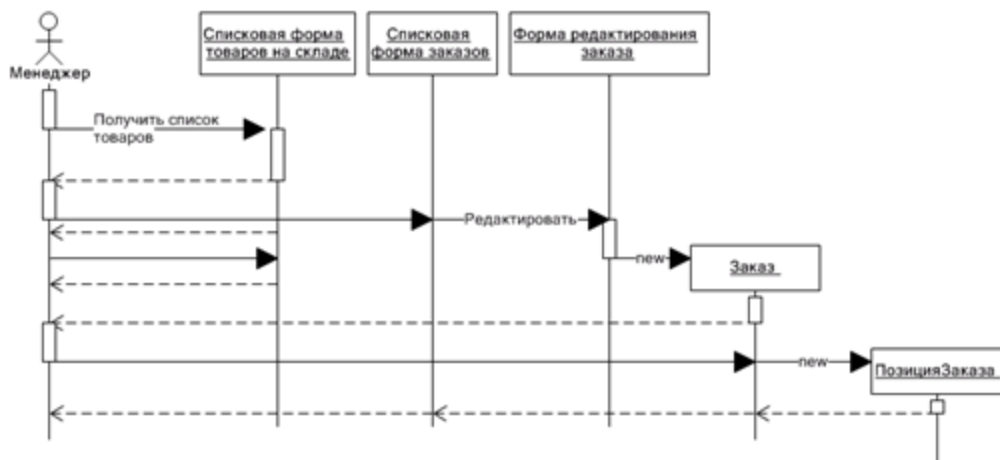
*Рис. 10.10. Диаграмма классов для банковских операций*

| Диаграмма    | Классов                                                                                      |
|--------------|----------------------------------------------------------------------------------------------|
| Содержит     | объекты, которые взаимодействуют в рамках сценария                                           |
|              | сообщения, которыми они обмениваются                                                         |
|              | возвращаемые результаты, связанные с сообщениями                                             |
| Используется | для уточнения диаграмм прецедентов, более детального описания логики сценариев использования |
| Подходит     | для документирования проекта (с точки зрения сценариев использования)                        |

# 13. UML: Диаграмма последовательностей (Sequence diagram)

[Почитать](#)

Пример:



| Диаграмма    | Последовательности                                                                           |
|--------------|----------------------------------------------------------------------------------------------|
| Содержит     | объекты, которые взаимодействуют в рамках сценария                                           |
|              | сообщения, которыми они обмениваются                                                         |
|              | возвращаемые результаты, связанные с сообщениями                                             |
| Используется | для уточнения диаграмм прецедентов, более детального описания логики сценариев использования |
| Подходит     | для документирования проекта (с точки зрения сценариев использования)                        |

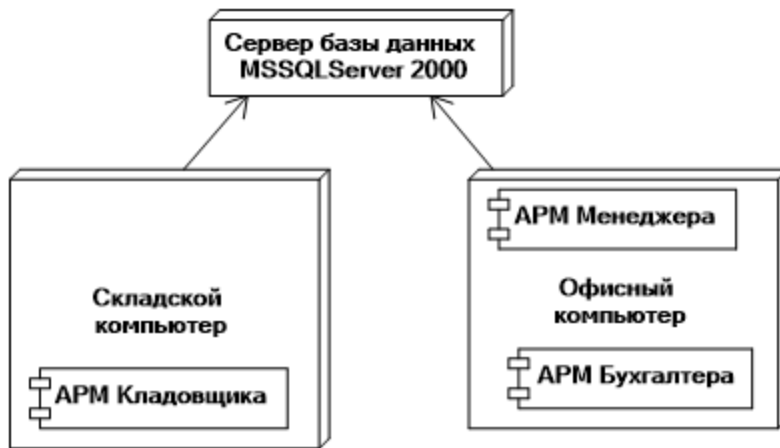
- Объекты обозначаются прямоугольниками с подчеркнутыми именами (чтобы отличить их от классов).
- Сообщения (вызовы методов) - линиями со стрелками.
- Возвращаемые результаты - пунктирными линиями со стрелками.

## 14. UML: Диаграмма размещения (Deployment diagram)

### Почитать

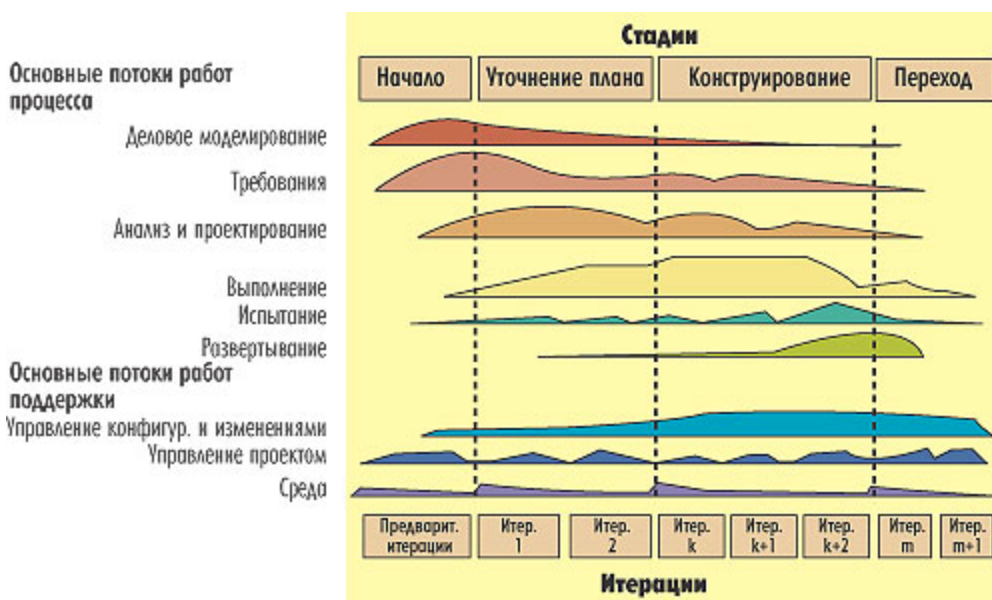
В UML развертывание – это процесс распределения артефактов по узлам или экземплярам артефактов по экземплярам узлов.

Пример:



| Диаграмма    | Размещения                                                   |
|--------------|--------------------------------------------------------------|
| Содержит     | графическое представление IT-инфраструктуры                  |
|              | топологию системы и распределение компонентов, их соединения |
| Используется | для аппаратно-программных систем                             |
| Подходит     | для рационального распределения компонент                    |
|              | для решения вспомогательных задач (например: безопасность)   |

## 15. \*UP методологии (90-е). RUP: основы процесса.



## 16. RUP: Фаза «Начало».

Каждая фаза заканчивается вехой - переход на следующий этап.

Цели:

- Определение границ проекта
- Описание основных сценариев использования системы
- Разработка технологических решений
- Оценка рисков
- Подготовка окружения
- Расчет стоимости!

Веха Lifecycle Objects:

- Согласие сторон в оценке сроков, первоначальной стоимости, требованиях, приоритетах, технологиях
- Оценены риски и выбраны стратегии смягчения последствий

| Фаза              | Начало               |
|-------------------|----------------------|
| Направление работ | Оценка проекта       |
| Артефакт          | Концепция ("Vision") |
| Веха              | Lifecycle Objects    |

## 17. RUP: Фаза «Проектирование» (уточнение).

стр. 23

**Исполняемая архитектура** — полностью законченные на базе выбранных технологий несколько характерных функций разрабатываемой системы.

Тестирование на этом этапе нужно для проверки основных **нефункциональных требований** (например пропускная способность)

- Документирование требований (включая детальное описание для большинства прецедентов).
- Сниженные основные риски.

Цели:

- Финализировать базовую архитектуру

- Разработать прототип
  - Убедиться, что архитектура, планы и сроки стабильны, риски учтены
  - Продемонстрировать, что архитектура подходит под требования
- Bexa Lifecycle Architecture – веха стабильности

Bexa Lifecycle Architecture:

- Концепция, требования, архитектура проекта стабильны?
- Сформированы критерии тестирования прототипов?
- Тестирование прототипов показало отсутствие основных рисков?
- Планы разработки подробны и приемлемы по цене?
- Решают вопрос с деньгами

| Фаза              | Проектирование                                                                                      |
|-------------------|-----------------------------------------------------------------------------------------------------|
| Направление работ | Разработка и тестирование                                                                           |
| Артефакт          | Спроектированная, реализованная и оттестированная исполняемая архитектура, Экономическое соглашение |
| Веха              | Lifecycle Architecture                                                                              |

Заказчик может менять соглашение по деньгам.

## 18. RUP: Фаза «Построение».

В фазе «Построение» происходит реализация большей части функциональности продукта.

Фаза Построение завершается первым внешним релизом системы и вехой начальной функциональной готовности (Initial Operational Capability).

Цели:

- "Быстро, дешево, сердито"
- Итеративно и инкрементально провести анализ, проектирование, разработку и тестирование
- Подготовить продукт и пользователей

Фаза Initial Operational Capability:

- Достаточно ли стабилен релиз?

- Все стороны готовы к релизу для юзеров
- Все еще норм по деньгам?

| Фаза              | Построение                     |
|-------------------|--------------------------------|
| Направление работ | Усиленная разработка           |
| Артефакт          | альфа- и бета-версии           |
| Веха              | Initial Operational Capability |

## 19. RUP: Фаза «Внедрение».

В фазе «Внедрение» создается финальная версия продукта и передается от разработчика к заказчику.

Это включает в себя программу бета-тестирования, обучение пользователей, а также определение качества продукта.

В случае, если качество не соответствует ожиданиям пользователей или критериям, установленным в фазе Начало, фаза Внедрение повторяется снова. Выполнение всех целей означает достижение вехи готового продукта (Product Release) и завершение полного цикла разработки.

Цели:

- Бета-тестирование
- Запуск маркетинга и продажи
- Отладка процессов устранения сбоев
- Открытый анализ соответствия ПО требованиям

Веха Product Release:

- Пользователи довольны?
- Че там с деньгами?

| Фаза              | Внедрение                    |
|-------------------|------------------------------|
| Направление работ | Тестирование качества, релиз |
| Артефакт          | Релиз                        |

| Фаза | Внедрение       |
|------|-----------------|
| Веха | Product Release |

## 20. Манифест Agile (2001).

Мир изменчив, бизнесу важно иметь возможность оперативно реагировать на изменчивый мир, внося правки (какая-то фича срочно нужна вчера).

**RUP такой возможности не давал.**

- **Люди и взаимодействия** важнее процессов и инструментов.
  - **Работающий продукт** важнее исчерпывающей документации.
  - **Сотрудничество с заказчиком** важнее согласования условия контракта.
  - **Готовность к изменениям** важнее следования первоначальному плану.
- не отрицая важности правого, мы больше ценим левое

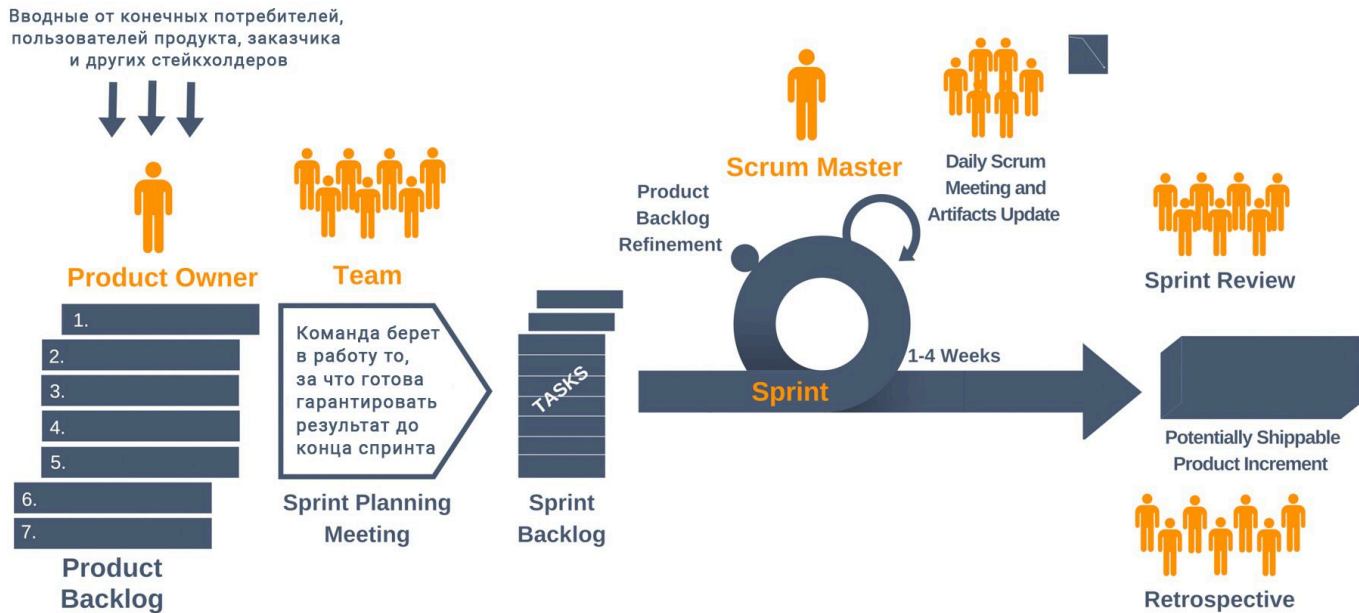
## 12 принципов AGILE:

- Удовлетворение требований заказчика
- Изменения требований приветствуются
- Частые выпуски программного продукта
- Ежедневная совместная работа
- Мотивированные профессионалы
- Непосредственное общение
- Работающий продукт -- показатель прогресса
- Постоянный ритм
- Техническое совершенство
- Простота
- Самоорганизующиеся команды
- Систематическая коррекция

## 21. Scrum.

Скрам -- фактическая реализация принципов Agile.

# МЕХАНИКА SCRUM-ПРОЦЕССА



**Product owner** - знает, что нужно сделать и в какой последовательности.

**Product backlog** - высокоуровневые требования к ПО, которые нужно реализовать.

**Sprint backlog** - детализированный backlog на 1 спринт (1-4 недели).

**Daily scrum** - daily meet up

**Scrum master** - опытный парень, следит за процессом, помогает с декомпозицией требований (product backlog -> sprint backlog)

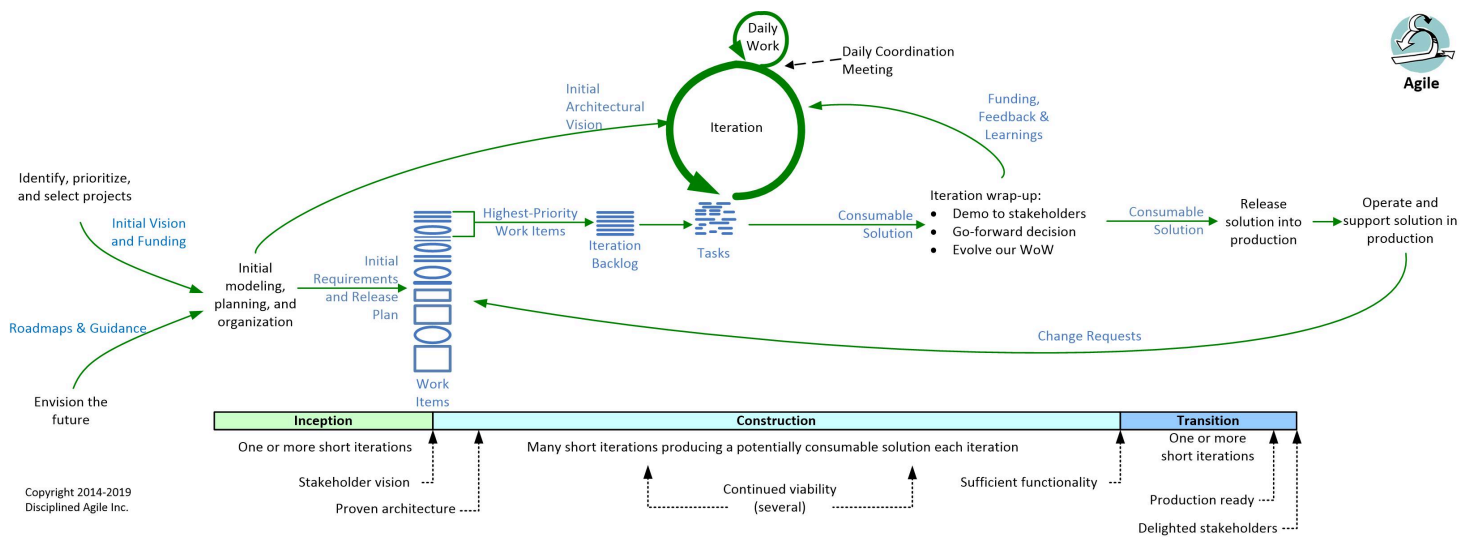
## 22. Disciplined Agile 2.X (2013).

Развитие Scrum привело к попытке совместить преимущества RUP со Scrum.

Инкрементальная модель разработки опирается на модель спринтов.

Подразумевается, что disciplined agile можно натянуть на большое-большое предприятие (в отличие от Scrum).

**Взяв плюшки от RUP получаем масштабируемость**



## 23. Требования. Иерархия требований.

- Требования -- условия или возможности, которым должна соответствовать система.
- Требование -- подробное описание того, что должно быть реализовано.
- Требование не описывает, как его необходимо реализовывать.

Требования описываются в документе **Software Requirements Specification** (как на 1-ой лабе в отчете).

Важно исключить разночтения требований, чтоб потом не попасть на бабос.

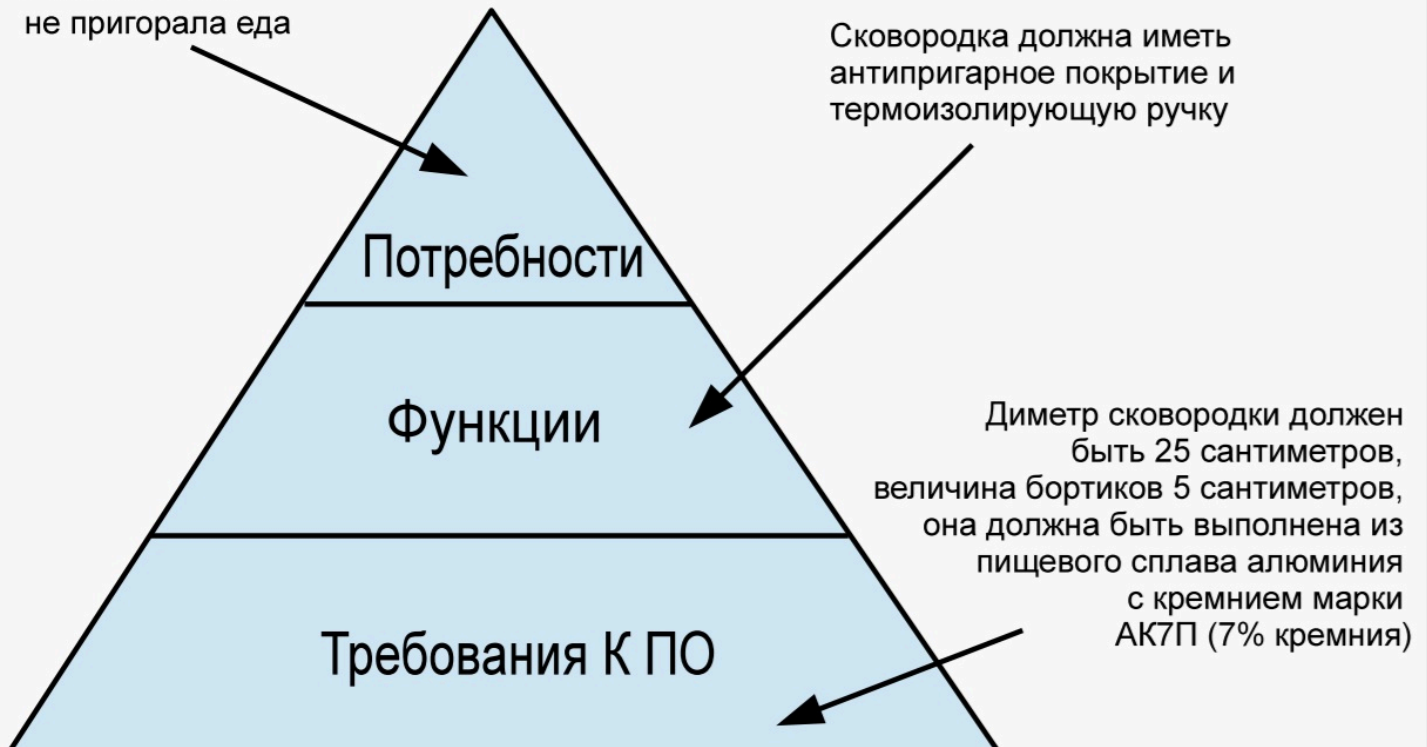


ИТМО ВТ

# Иерархия требований

Мне нужна сковородка,  
и чтобы на ней  
не пригорала еда

Сковородка должна иметь  
антипригарное покрытие и  
термоизолирующую ручку



## 24. Свойства и типы требований (FURPS+).

- Корректность
- Однозначность
- Полнота
- Непротиворечивость
- Приоритизация
- Проверяемость
- Модифицируемость
- Отслеживаемость

FURPS — классификация требований к программным системам.

Образована от первых букв слов:

- Functionality — Функциональные требования: свойства, возможности, безопасность.  
Являются основными, по этим требованиям строятся диаграммы вариантов использования (Use case diagram).

- Usability — Требования к удобству использования (UX): человеческий фактор, эстетика, последовательность, документация.
- Reliability — Требования к надежности: частота возможных сбоев, отказоустойчивость, восстанавливаемость, предсказуемость устойчивости.
- Performance — Требования к производительности: время отклика, использование ресурсов, эффективность, мощность, масштабируемость.
- Supportability — Требования к поддержке: возможность поддержки, ремонтпригодность, гибкость, модифицируемость, модульность, расширяемость, возможность локализации.

## 25. Формулирование требований. Функциональные требования.

{id}{система}должна/shall{требование}

- id -- уникальный идентификатор в системе контроля и изменения требований
- система -- имя системы или подсистемы.
- должен/shall -- юридически обязывающее слово.
- требование -- требование, которое должно быть исполнено.

Функциональные требования определяют:

- Feature sets -- наборы функциональных требований
- Capabilities -- возможности ПО
- Security - безопасность

## 26. Требования к удобству использования и надежности.

Usability:

- Human factors -- учет особенностей пользователей (плохое зрение, слух).
- Aesthetics -- эстетические требования. (Толстый брендбук)
- Consistency in the user interface -- согласованность пользовательского интерфейса.
- Online and context-sensitive help -- требования к справочной подсистеме.
- Wizards and agents -- мастера (пошаговая штукавина) и ПО, повышающие продуктивность и простоту работы пользователя.
- User documentation -- требования к пользовательской документации.

- Training materials -- требования к учебным материалам.

Reliability:

- Frequency and severity of failure -- частота и обработка отказов.
- Recoverability -- способность системы восстанавливать своё продуктивное состояние.
- Predictability -- предсказуемость поведения
- Availability -- готовность системы к решению задач
- Accuracy -- точность
- MTBF -- среднее время между отказами
- Коэффициент готовности -- связь между средним временем между отказами и временем восстановления системы.

## **27. Требования к производительности и поддерживаемости.**

Performance:

- Speed -- скорость решения задач
- Efficiency -- эффективность
- Throughput -- пропускная способность
- Response time -- время отклика
- Recovery time -- время восстановления
- Resource usage -- использование системных (и других) ресурсов  
(все эти требования связаны и задаются сообща, а не по отдельности)

Supportability:

- Extensibility -- расширяемость
- Adaptability -- адаптируемость под конкретные задачи
- Maintainability -- поддерживаемость
- Compatibility -- совместимость
- Configurability -- способность задавать конкретную конфигурацию
- Serviceability -- возможность проведения профилактик и обслуживания
- Installability -- требования к установке на разные системы
- Localizability -- локализуемость для разных для разных языком и географических регионов

## 28. Атрибуты требований.

### 1. Приоритет -- MoSCoW

- MUST have (Minimum Usable Subset) -- фундаментальные для системы требования
- Should have -- важные
- Could have -- потенциально возможные, улучшающие, к примеру пользовательское отношение
- Won't have (Would like to have) -- мб в следующей версии

### 2. Статус:

- предложенные, одобренные, отклоненные, включенные

### 3. Трудоемкость:

- человеко-часы (чаще всего), функциональные точки, use-case points, попугаи

### 4. Риск

### 5. Стабильность:

- высока, средняя, низкая

### 6. Целевая версия:

- когда включим в продукт?

## 29. Описание прецедента.

Прецеденты(case) могут быть описаны в виде сценариев или, как, сейчас принято(модно) - user story.

В дальнейшем разработчики зачастую анализируют не сколько требования, сколько сценарии.

use-case description in RUP

| Прецедент: PieSelling                                                                                                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>ID:</b> 2                                                                                                                                                                                                                            |
| <b>Краткое описание:</b> Бабушка продаёт пирожки.                                                                                                                                                                                       |
| <b>Главные актёры:</b> Бабушка-продавец, Клиент (или любой другой пользователь).                                                                                                                                                        |
| <b>Второстепенные актёры:</b> нет.                                                                                                                                                                                                      |
| <b>Предусловия:</b><br>Клиент знает, какой пирожок он предпочитает.<br>Бабушка проверила весь ассортимент.<br>У клиента достаточно средств.                                                                                             |
| <b>Основной поток:</b><br>1. Клиент обращается к Бабушке за пирожками.<br>1.1. Клиент называет количество и номенклатуру пирожков.<br>1.2. ALT1 PieRecomendation.<br>1.3. Бабушка подтверждает номенклатуру и называет общую стоимость. |

## 30. Риски. Типы Рисков.

Риск — потенциально опасный для проекта фактор.

Риски бывают:

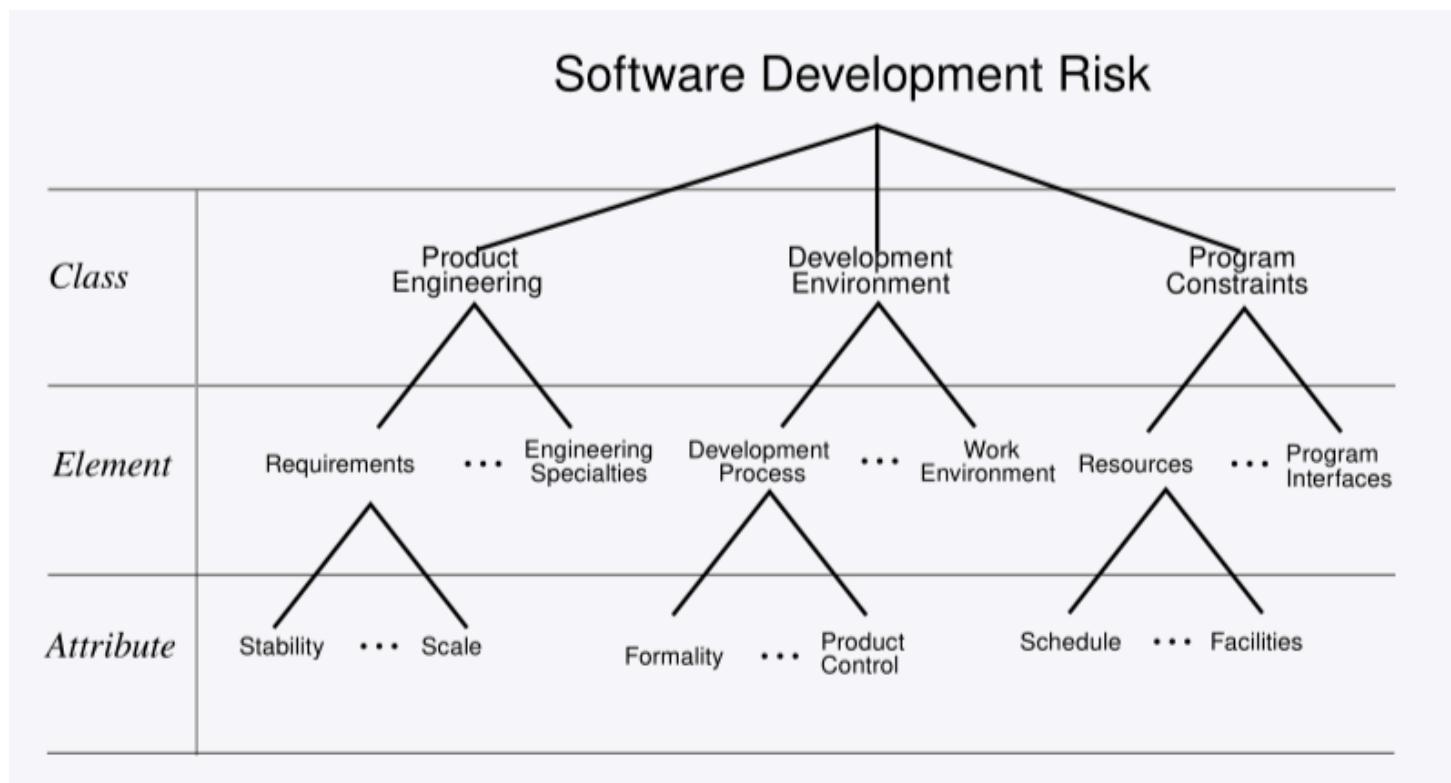
- Прямые
  - можем в ясном виде управлять: уменьшать вероятность возникновения, реагировать
- Непрямые
  - зависят от внешних факторов
  - не поддаются управлению

| Тип                          | Характеристика / Пример                          | Чья компетенция |
|------------------------------|--------------------------------------------------|-----------------|
| Ресурсные (управляемые)      | Организационные, финансовые, <b>люди</b> , время | Менеджеры       |
| Бизнес-риски (неуправляемые) | Конкуренция, подрядчики, убыточность решения     | Стейкхолдеры    |
| Технические (упр)            | Границ проекта, технологические, внешние API     | Разработчики ПО |
| Политические (неупр)         | Сферы влияния менеджеров                         |                 |
| Форс-мажор (неупр)           |                                                  |                 |

## 31. Управления рисками. Деятельности, связанные с оценкой.

Что можно/нужно делать для снижения рисков?

### Идентификация рисков



Классы рисков -> Элементы -> Атрибуты, которые указывают на возможные источники возникновения рисков.

## Анализ рынка

- Различные модели и методы анализа: стоимостной, сетевой, качественной факторов
  - так как работа осуществляется людьми, то данные параметры обычно задаются нечетко
- Вероятность и масштаб потерь

## Приоритизация рисков

- Risk Exposure:  $RE = Prob(U0) * Loss(U0)$  - произведение вероятности наступления риска на величины денежных потерь
- Другие факторы анализа
- Создается документ "ТОП-10 рисков"

## 32. Управления рисками. Деятельности, связанные контролем и управлением.

### Планирование реакции

- Избегание риска, реорганизация проекта
- Перенос риска (на кого-то)
- Сокращение вероятности риска -> Использование более проверенного стороннего ПО
- Прием риска -> Проработка рисков заранее

### Разрешение риска

- Прототипы системы
- Моделирование и симуляция
- Различные индикаторы
- Аналитическая работа
- Подбор персонала

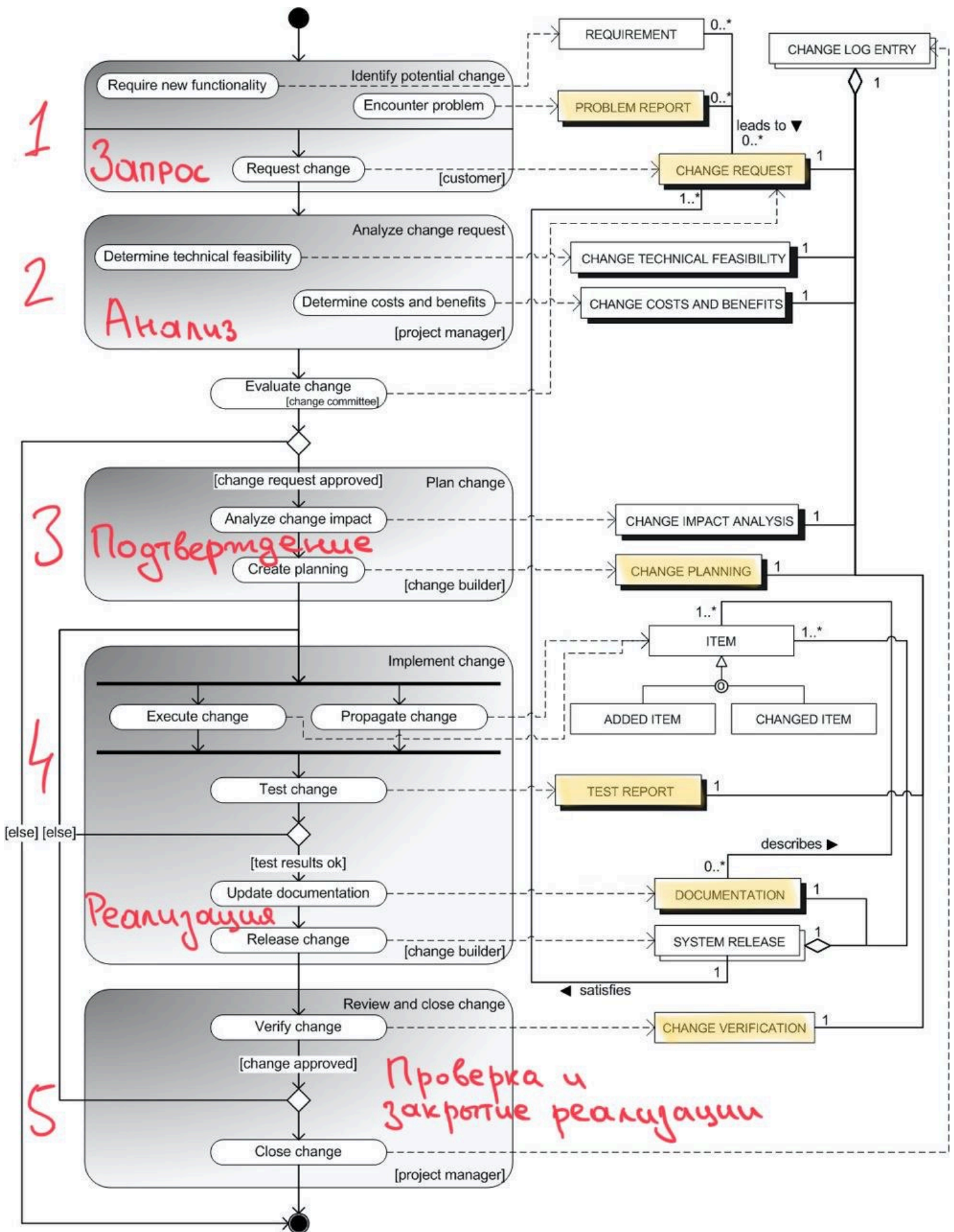
### Мониторинг рисков

- Во время итераций разработки

- ТОП-10 список
- Переоценка рисков
- Автоматизация системы
  - Числовые параметры
  - Автоматический сбор из вспомогательных систем: Контроль версий, управление задачами + плагины Jira

## **33. Изменение. Общая модель управления изменениями.**

Изменение — контролируемое, журналируемое обновление системы.



## 34. Системы контроля версий. Одновременная модификация файлов.

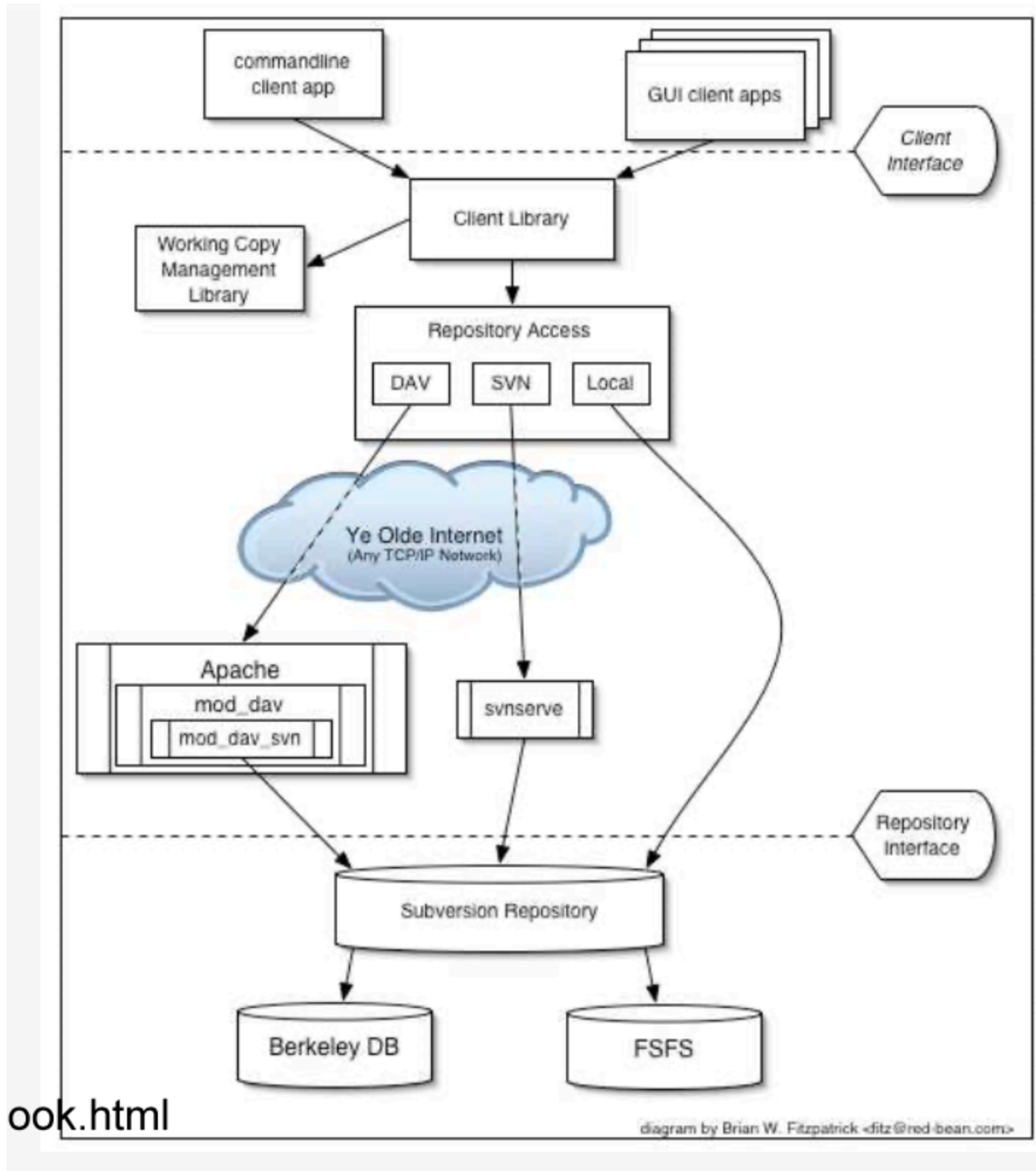
### Системы контроля версий

- Управляют изменениями в коде
- Поддерживают групповую работу
- Основные типы:
  - На основе файловой системы
  - Централизованные (svn)
  - Распределенные (git)

### Одновременная модификация файлов

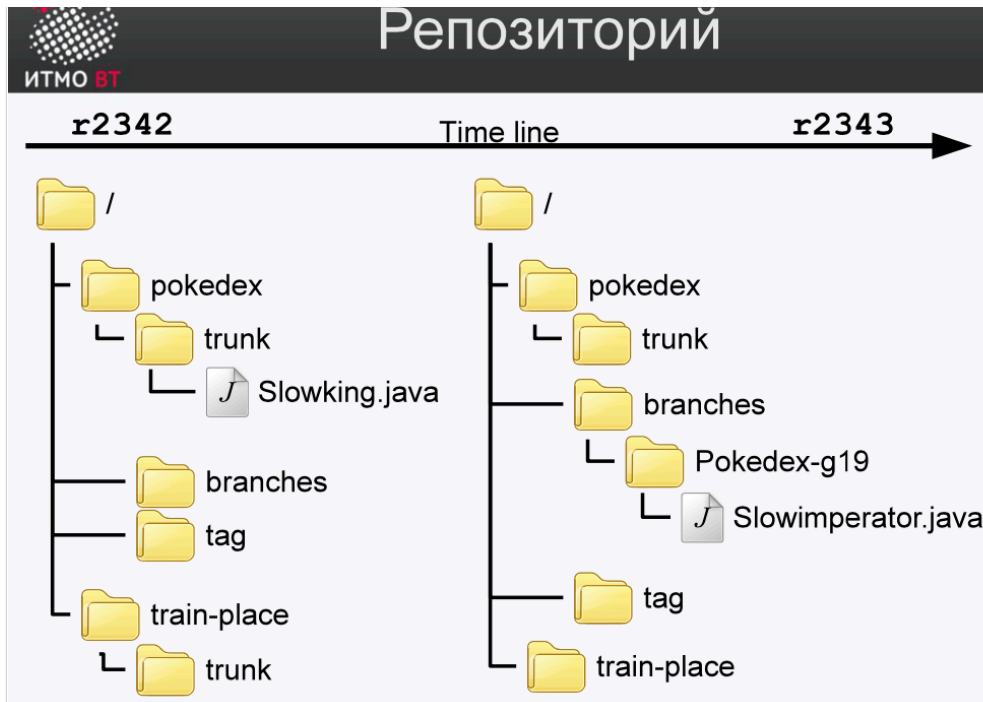
- Lock-modify-unlock:
  - В основном в VCS на файловых системах
  - Замедляет работу команду
- Copy-modify-merge:
  - Своя рабочая копия у каждого
  - Трудности слияния

## 35. Subversion. Архитектура системы и репозиторий.



1. Хранение самого уровня репозитория, который лежит или в Berkeley DB или FSFS

2. Доступ к репозиторию - svnserve, демон, через который отдаем команды
3. Удаленный доступ - svn+https и ssh+svn
4. Также управление локальными копиями файлов, которые изменяет сам разработчик



Репозиторий — это набор файлов проекта(ов), организуемый определённым иерархическим образом для удобной работы с проектом. На слайде показан пример репозитория.

В SVN каждая фиксация изменений (svn commit), пришедшая в репозиторий, повышает версию этого репозитория на 1. Ревизия репозитория — это всегда целое число, уникально характеризующее набор файлов, лежащих в репозитории на какой-то момент времени, задаваемый номером ревизии. В каждый момент времени, при условии успешной фиксации изменений, репозиторий является целостным.

Каждая фиксация изменений svn обязана включать файлы, изменения которых должны быть помещены в репозиторий. Если такое изменение производится из командной строки, то эта строка должна содержать все файлы. В одну фиксацию изменений рекомендуется помещать логически законченное обновление функционала, или, например, исправление одного дефекта. Не рекомендуется объединять разные изменения в один коммит.

Организация файлов обычно такова: существуют каталоги проектов (здесь pokedex и train-place), внутри которых размещены директории, предназначенные для организации работы с разными версиями проекта.

В каталоге trunk происходит основной процесс разработки. В этот каталог разработчики ежедневно вносят результаты своего труда, оформленные в виде целостного набора изменений (с точки зрения и набора файлов, и их логического содержимого). В случае необходимости поставки различных версий разным заказчикам, либо с целью фиксации истории релизов, стабильные на некоторый момент времени копии могут формироваться на основе содержимого каталога trunk и копироваться в дополнительные каталоги branch и tags.

Ветки:

trunk - основная ветка разработки

branches - все остальные ветки проекта ( + модификации сторонних библиотек)

tag - релиз версия продукта или каталог с минорной версией для исправления дефектов

## 36. Subversion: Основной цикл разработчика. Команды.

**Основной цикл разработчика**  
ИТМО BT

**svn checkout** — первоначальное создание рабочей копии.

Ночь! Вечер День Утро  
↓

- Обновить рабочую копию:
  - **svn update**
- Изменить необходимые файлы:
  - **svn (add, delete, copy, move, mkdir)**
  - Просмотр изменений: **svn status** и **svn diff**
  - Откат изменений: **svn revert**
- Фиксация изменений на сервере:
  - **svn update** для загрузки изменений других.
  - Разрешить конфликты содержимого и структуры файлов.
  - **svn commit** — собственно фиксация.

Основной цикл разработки использует описанные выше свойства SVN.

В начале работы сервер содержит в рабочей ветке все последние накопленные обновления других разработчиков, и в вашем каталоге существует локальная версия предыдущего дня. Чтобы начать ежедневный цикл работы, необходимо скачать с сервера себе в файловую систему обновление локальной копии. При этом скачиваются не только сами данные, но и метаданные. Управляющая информация нужна для обеспечения целостности и для определения того, куда будет производиться коммит. Таким образом, утром рабочего дня каждый разработчик должен выполнить **svn update** и привести свою рабочую копию до того состояния, которое есть на сервере.

Затем производится "великое творение кода" и меняются необходимые файлы. Используются такие команды, как **svn add** (для создания (добавления) файла), **svn delete** (для удаления файла), **svn copy**, и другие (то же самое касается и директорий, а также перемещения файлов внутри рабочей копии).

Если нужно провести откат изменений, используется команда **svn revert**, а команды **svn status** и **svn diff** помогают ориентироваться в изменениях по сравнению с сервером.

Вечером производится фиксация изменений на сервере. Так как другие разработчики тоже могли внести изменения, с сервера снова скачивается версия, там находящаяся. При этом существует вероятность возникновения конфликтов между изменениями данного разработчика и изменениями, сделанными его коллегами. Данные конфликты необходимо разрешить. После этого производится собственно фиксация с помощью команды **svn commit**.

При этом, тому, кто делает первый коммит, не нужно разрешать каких-либо конфликтов, так как он загружает на сервер первое изменение. Также следует отметить, что конфликты между версиями разных разработчиков могут привести и к конфликтам внутри команды. Существует также понятие ночного билда (Nightly build), который собирается именно на основе вечерних коммитов.

## 37. Subversion: Конфликты. Слияние изменений.

В случае слияния файлов часто случаются проблемы в содержимом файлов(те внесенная разработчиками информация противоречит друг другу(условно)).

В svn есть различные способы решения конфликтов:



# Конфликты содержимого файлов

- (e) edit — изменить файл в редакторе.
- (df) diff-full — показать список несовпадений между версиями.
- (r) resolved — подтвердить, что конфликт разрешен в текущей (м.б. отредактированной) версии файла.
- (dc) display-conflict — показать все конфликты.
- (mc) mine-conflict, (tc) theirs-conflict — подтвердить мои (или из репозитория) версии для всех конфликтов.
- (mf) mine-full, (tf) theirs-full — подтвердить версии полностью для файла.
- (p) postpone — отложить «на потом».
- (l) launch — запустить внешнее средство.

```
$ svn update
Updating '.':
U    Pokedex.xml
G    Slowimperator.java
Conflict discovered in 'Slowking.java'
Select:
```

filename.mine — до update  
filename.rOLDREV — до правок  
filename.rNEWREV  
— в репозитории

Также стоит учитывать, что в IDE обычно имеются средства GUI, в которых можно удобно просматривать конфликты содержимого файлов

В случае, если нам требуется слить изменения из различных веток, хорошей практикой будет использование команды `svn merge`

## 38. GIT: Архитектура и команды.

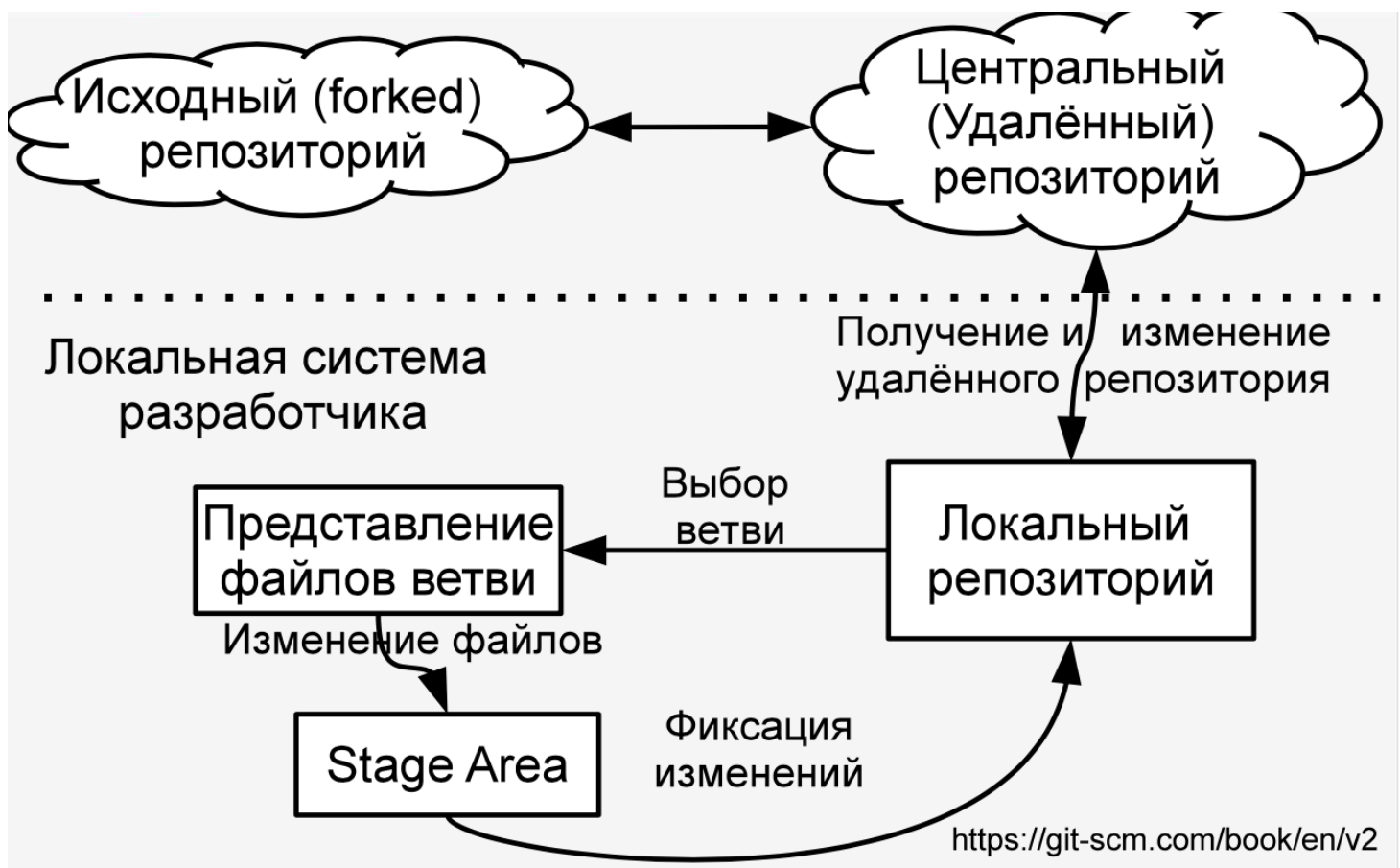
GIT - распределенная система контроля версий

Особенность архитектуры GIT состоит в присутствии локальной копии проекта у каждого из разработчиков,

а также возможности выборочного применения доступных изменений

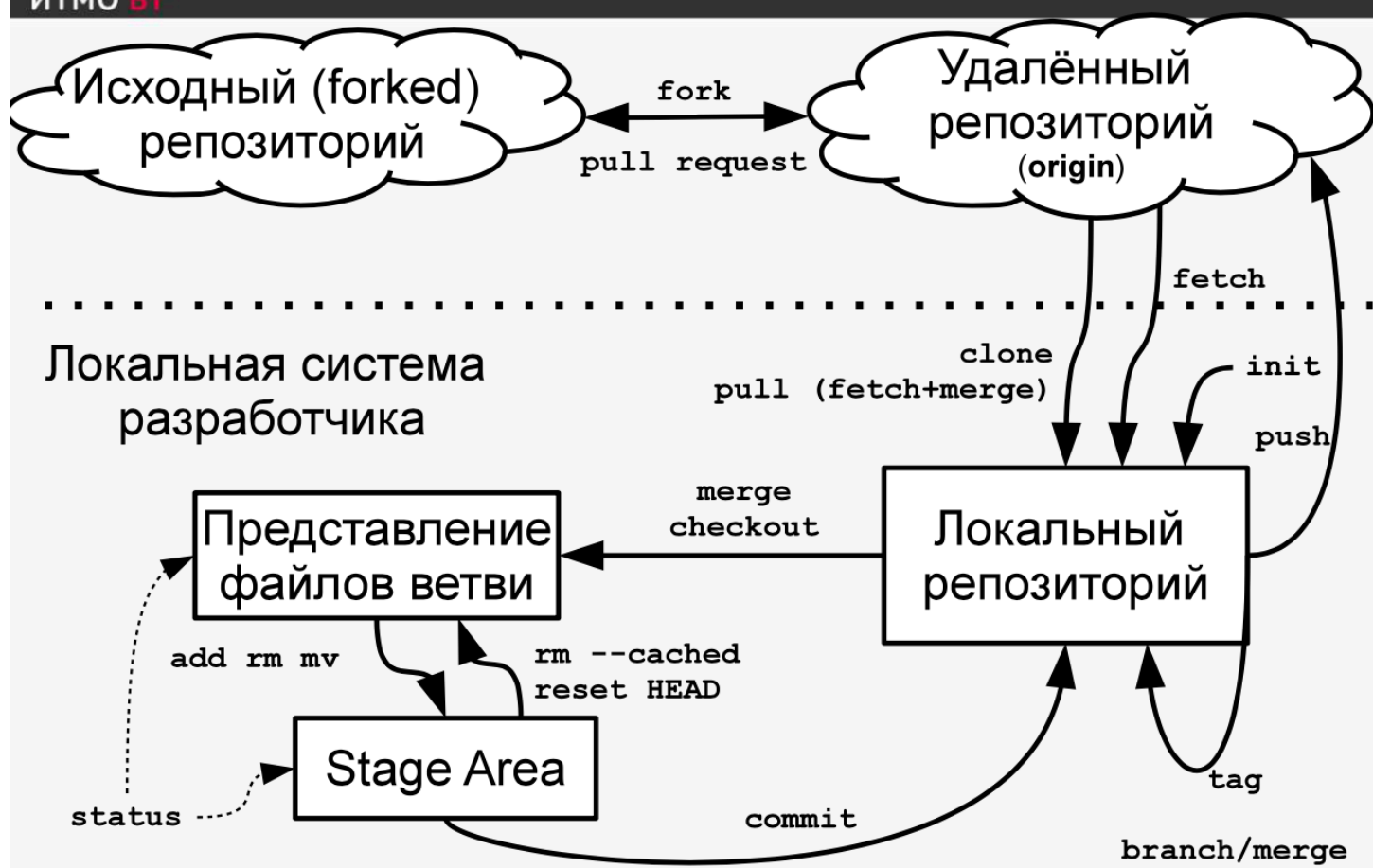
Исходя из этого, рабочий процесс строится на основании одной из трех стратегий.

1. централизованный рабочий процесс - по сути как в SVN
2. Рабочий процесс с менеджером по интеграции - каждый дев работает отдельно, далее коммитит, и далее есть отдельный мужик, который занимается слиянием всего
3. Рабочий процесс с диктатором и лейтенантами - аналог принципа работы пункта №2, но есть умный чел, репозиторий которого выступает как эталонный



Список команд от Клименкова:

# GIT: основные команды



## 39. GIT: Организация ветвей репозитория.

Философия GIT в том, что каждое отдельное изменение должно разрабатываться в отдельной ветви.

**Master** - в данной ветке находятся основные версии продукта

**Develop** - в данную ветку все разработчики сливаются свои изменения, связанные с функциональностью продукта, далее ветка перетекает в Master.

**Feature** - эти ветки создаются как подмножества Developer. Туда аккумулируются коммиты и после завершения разработки функционала, ветка сливается в Develop.

**Release** - после того, как разработка на Develop завершена, идет тщательная проверки и данная ветка используется для

внесения изменений, которые мешают выпуску новой версии. Далее все сливается в Develop и процесс на ветке запускается заново.

Hotfix - ветка для критических исправлений по ходу работы у пользователя. Изменения с нее сливаются как в Master, так и в Develop.

## 40. GIT: Плагин git-flow.



Каждое проделываемое в Git действие над рабочей копией (создание ветвей, их слияние и т.п.) требует достаточно большого количества команд, и может вызвать у разработчиков сложности с синтаксисом команд. Для того, чтобы упростить эту работу, был создан плагин Git flow, позволяющий работать с версиями именно в терминах версий, а не операций, без точного знания команд и последовательностей действий.

Для использования Git flow необходима определённая начальная настройка репозитория. Для этого у Git flow есть функция `init`, которая при старте задаёт пользователю несколько вопросов и производит необходимую настройку.

Для создания новой `feature` в git flow используется команда `git flow feature start FEATURE_NAME`. Когда надо указать на окончание изменений, используется `git flow feature finish FEATURE_NAME`. Остальные команды формируются аналогичным образом. Все это позволяет пользоваться репозиторием на более высоком, независимом от синтаксиса git уровне, при этом реализуется модель, указанная на предыдущих слайдах.

Git Flow - расширение Git. Философия заключается в работе над версионированием в терминах релиза версий, а не самих операций

## 41. Системы автоматической сборки: предпосылки

## ПОЯВЛЕНИЯ

1. Рутинный процесс сборки
2. Отличия архитектуры информационных систем
3. Медленная сборка

Если брать параллельные процессы компиляции, то возникает проблема доступа сборочных узлов к общим исходным кодам проекта

Вышеперечисленные причины и привели к появлению разнообразных систем сборки

## 42. Системы сборки: Make и Makefile.

Make - система сборки, появившаяся впервые в Unix-системах, существует и по сей день.

Является императивной(условно) системой сборки. Исходный код сборки располагается в Makefile, который

в свою очередь располагается в корневой директории проекта. Далее в корневой директории прописывается

команда "make" и происходит сборка проекта. По умолчанию основной направленностью make являются "file targets".

Чтобы сказать Make, что мы хотим работать не с файлами, а с наименованием цели, используем слово "PHONY".

Основная цель - all. Пример цели:

```
all:
    gcc -o main main.c
```

## 43. Системы сборки: Ant. Команды Ant.

Apache Ant представляет собой императивную систему сборки, использующую в качестве основного файла build.xml.

Разрабатывалась для использования в Java-проектах. Файл состоит из target, в которых описана определенная

последовательность действий для достижения поставленной цели. Пример простого build.xml.

```

<project name="MyProject" default="dist" basedir="../../../../..">
  <property name="src" location="src"/>
  <property name="build" location="build"/>

  <target name="init">
    <mkdir dir="${build}"/>
  </target>

  <target name="compile" depends="init">
    <javac srcdir="${src}" destdir="${build}"/>
  </target>
</project>

```

## 44. Системы сборки: Ant-ivy.

Apache Ivy - транзитивный менеджер пакетов, который используется совместно с Apache Ant. Пакет скачивает из Maven-репозитория(определение см ниже). Для конфигурации использует файлы ivysetting.xml и ivy.xml.

В головной build.xml подключается при помощи:

```

<project name="MyProject" xmlns:ivy="antlib:org.apache.ivy.ant" default="dist" basedir=". ">
  <target name="resolve">
    <ivy:retrieve/>
  </target>

```

Пример зависимости:

```

<ivy-module version="2.0">
  <info organisation="org.apache" module="hello-ivy"/>
  <dependencies>
    <dependency org="jakarta.annotation" name="jakarta.annotation-api" rev="1.3.5" />
  </dependencies>
</ivy-module>

```

## 45. Системы сборки: Maven. POM. Репозитории и зависимости.

Apache Maven - одна из самых популярных систем сборки для Java-проектов на сегодняшний день. Использует декларативный подход. Основным файлом выступает POM, (Project Object Model)(основной файл - pom.xml)

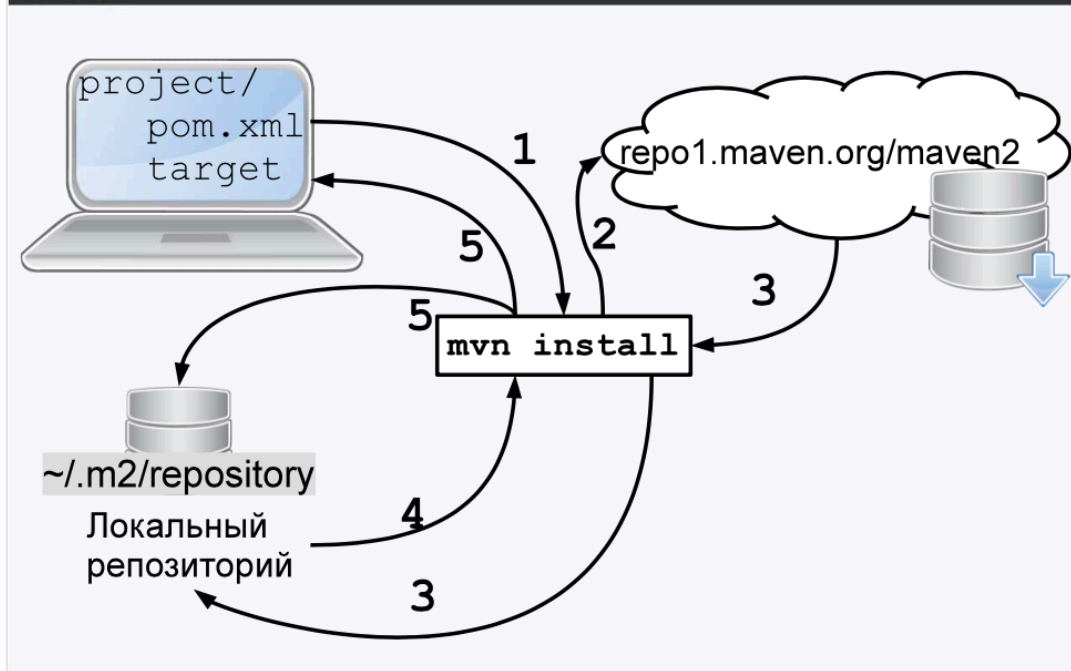
```
<project xmlns="http://maven.apache.org/POM/4.0.0"      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>
    <groupId>com.javatpoint.application1</groupId>
    <artifactId>my-app</artifactId>
    <version>1</version>

</project>
```

В POM указываются имя, версия и тип программы, местоположение исходных кодов проекта зависимости, плагины, альтернативные конфигурации проекта

Репозитории и зависимости.



На слайде представлен принцип работы репозитория Maven.

Цель по умолчанию у Maven — `install`. Её назначение состоит в том, чтобы в локальном репозитории пользователя была собрана версия продукта.

Продукт должен быть собран в директории `target` и сохранён в локальном репозитории. Сначала (1) Maven читает файл `pom.xml` с информацией о проекте и определяет зависимости проекта от внешних программных продуктов. Эти зависимости также декларативно описаны в `pom.xml`.

После этого Maven скачивает необходимые для сборки файлы проекта с удалённого репозитория (2) и сохраняет их в локальный репозиторий (3) до тех пор, пока не скачает все внешние зависимости. Затем осуществляется сама сборка (4). В итоге готовый, собранный продукт помещается в директорию `target` и в локальный репозиторий (5), после чего возможно помещение продукта в удалённый репозиторий.

Все цели в Maven частично скриптовые, а частично описаны внутри кода модулей, на основе своих POM.

По умолчанию производится обычная последовательная сборка кода. При необходимости её можно распараллелить.

## 46. Maven: Структура проекта. GAVNO.



### Структура проекта

- target — рабочая и целевая директории.
- src/main — основные исходные файлы:
  - src/main/java — исходные файлы java.
  - src/main/webapp — web-страницы, jsp, js, css...
  - src/main/resources — файлы, которые нет необходимости компилировать.
- src/test — исходные файлы для тестов:
  - src/test/java
  - src/test/resources

В Maven существует система каталогов по умолчанию (src/main/java - исходные коды проекта) (src/main/resources - дополнительные файлы проекта, .properties файлы)

После работы готовые файлы кладутся в директорию target

GAV.

В Maven все зависимости описываются в GAV-синтаксисе, который представляет собой: groupId:artifactId:version. Так описывается любая внешняя зависимость. Данные зависимости скачиваются из интернета, поэтому если это не нужно, надо запретить в явном виде.

```
<project>  
  <modelVersion>4.0.0</modelVersion>  
  <groupId>com.mycompany.app</groupId>  
  <artifactId>my-app</artifactId>  
  <version>1</version>  
</project>
```

## 47. Maven: Зависимости. Жизненный цикл сборки. Плагины.

Зависимости см пункт выше



## «Жизненный» цикл

- generate-sources/generate-resources
- compile
- test-compile
- test
- package
- integration-test (pre and post)
- install
- deploy

В приведённом на слайде жизненном цикле Maven-приложения последовательно вызываются цели и стадии его сборки.

Фаза generate sources (генерация исходных кодов) применяется тогда, когда в приложении часть исходного кода автогенерируется (например, на базе DSL-грамматики собирается компилятор предметно-ориентированного языка). После этого исходный код компилируется (фаза compile), а затем компилируются тесты на базе приложения (фаза test-compile).

После этого приложение тестируется (фаза test), собирается в пакеты (фаза package) и производится интеграционное тестирование (фаза integration-test). Приложение устанавливается в локальном репозитории (фаза install).

Последняя цель, deploy, вызывается, если указан сервер приложений, и заключается в установке приложения на этот сервер.



## Плагины

- Все операции над проектом выполняются плагинами.
- Плагины в качестве точек входа содержат цели, связанные с ЖЦ:
  - Core: clean compiler deploy failsafe install resources site surefire verifier.
  - Packaging: ear ejb jar rar war app-client/acr shade source verifier.
  - Reporting, Tools, and thirdparty.

<http://maven.apache.org/plugins/index.html>

Плагины в Maven используются для управления сборкой. Их надо в явном виде включать в модель POM.

Если в Ant все действия над приложениями надо в явном виде прописывать, то в Maven достаточно декларативных высказываний.

Вызов плагинов управляется общей логикой сборки проекта. Плагины в качестве точек входа содержат цели, связанные с ЖЦ сборки.

В число целей, например, входят цели упаковки (packaging) и цели, связанные с автоматической генерацией отчетов по разработке.

## 48. Системы сборки: Gradle. Преимущества и файл сборки.

Gradle — это современная система автоматической сборки с открытым исходным кодом, основанная на концепциях Apache Ant и Apache Maven. В отличие от XML-формата Maven, Gradle использует предметно-ориентированный язык (DSL) на основе Groovy или Kotlin для написания сценариев сборки.

Gradle 是一个基于 Apache Ant 和 Apache Maven 概念的开源构建自动化系统。它使用一种基于 Groovy 或 Kotlin 的领域特定语言（DSL）来编写构建脚本，而不是 Maven 的 XML 格式。

### Преимущества Gradle

- Высокая производительность
- Гибкость

- Лаконичный и мощный DSL 简洁且强大的 DSL
- Управление зависимостями 卓越的依赖管理
- Поддержка многопроектных сборок: Эффективно управляет зависимостями и порядком сборки в проектах с множеством модулей. 多项目构建支持

## Файл сборки

Файлы сборки Gradle существуют в двух форматах:

- `build.gradle` — на основе DSL Groovy (классический, динамический).
- `build.gradle.kts` — на основе DSL Kotlin (рекомендуемый официально, обеспечивает лучшую автодополнение и проверку типов в IntelliJ IDEA).

# 49. Системы сборки: GNU autotools. Создание конфигурации проекта.

Autotools, или система сборки GNU,— это набор программных средств, предназначенных для поддержки переносимости исходного кода программ между UNIX-подобными системами.

Требуемая последовательность команд для запуска: `./configure && make && make install`



### GNU autotools

- Основаны на макропроцессоре общего назначения m4.
- Исторически используется для распространения программ с открытым кодом.
- Пользователю нужно знать три команды:
  - `./configure; make; sudo make install`

```

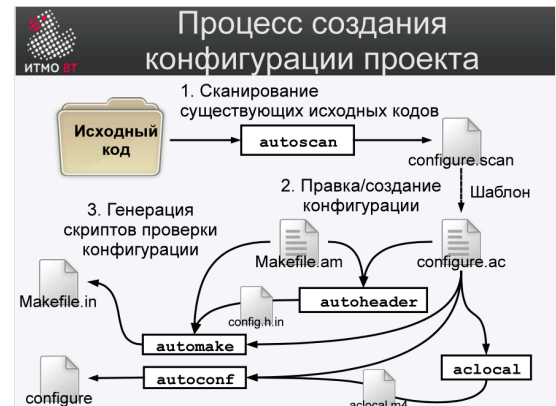
$ ./configure
checking whether we are using the GNU C compiler... yes
checking whether gcc accepts -g... yes
checking whether gcc understands -c and -o together... yes
checking for style of include used by make... GNU
checking dependency style of gcc... gcc3
checking for fork... yes
checking for daemon... yes

```

GNU autotools основаны на макропроцессоре общего назначения m4, существовавшего с момента написания первых версий Unix. Макропроцессор — это программа, преобразующая входной текст в выходной, руководствуясь правилами замены последовательностей символов, называемых правилами макроподстановки. Макроподстановки по принципу работы подобны директивам препроцессора в C.

GNU autotools исторически используется для распространения программ с открытым кодом. Для сборки системы из исходных материалов пользователю нужно знать всего три команды, применяемые последовательно: `./configure`, `make` и `sudo make install`. Этого достаточно для установки собранного продукта. На слайде приведен процесс конфигурации.

Стоит отметить, что GNU autotools полностью платформенезависимы.



Как разработать конфигурацию программного продукта для последующего использования на разных платформах?

С помощью утилиты `autoscan` последовательно сканируется существующий исходный код, и на основании имеющейся базы данных выделяются участки кода, которые могут зависеть от особенностей платформы и операционной системы. По результатам сканирования формируется шаблон конфигурационного файла `configure.scan`, который затем редактируется вручную. В результате получается файл `configure.ac`.

Также вручную создается файл `Makefile.am`. В нём в явном виде указываются названия исполняемых программ, из каких исходных файлов они должны быть собраны, и другие зависимости. Такой файл создаётся в каждом из подкаталогов исходных файлов.

После этого вызывается последовательность команд. Первая из них, `autoheader`, создаёт шаблон `config.h.in` для файла `config.h`, а команда `aclocal` проверяет, что установлено на локальной системе разработчика, и должно быть использовано в проекте.

Затем запускаются утилиты `automake` и `autoconf`, в результате чего в дистрибутиве появляются файлы `Makefile.in` и `configure`, которые позже используются при определении текущей конфигурации на целевой системе, где будет собираться ПО.

# 50. Системы сборки: GNU autotools. Конфигурация и сборка проекта.



Конфигурация **configure.ac** и зависимости **Makefile.am**

```
AC_PREREQ([2.68])
AC_INIT([tsconvd], [0.1], [support@tune-it.ru])
AC_CONFIG_SRCDIR([src/tsconvd.c])
AC_CONFIG_HEADERS([config.h])
AM_INIT_AUTOMAKE([-Wno-portability 1.10])
# Checks for programs.
AC_PROG_CC
CFLAGS=-g
# Checks for libraries.
AC_CHECK_FUNCS([fork daemon exit signal])
PKG_CHECK_MODULES(GLIB, [glib-2.0])
PKG_CHECK_MODULES(GIO, [gio-2.0])
PKG_CHECK_MODULES(GST, [gstreamer-0.10])

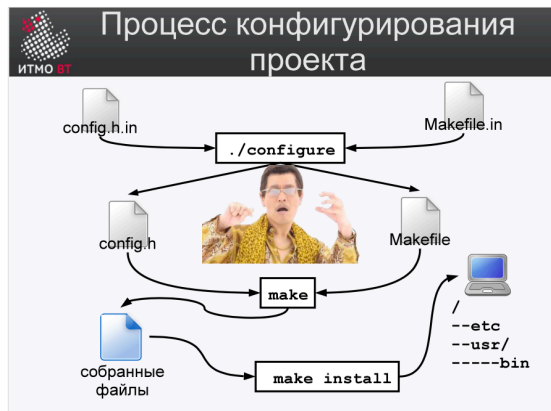
tsconvd_SOURCES = tsconvd.c gstreamer.c
bin_PROGRAMS = tsconvd
tsconvd_CFLAGS = -g $(GST_CFLAGS) $(GIO_CFLAGS)
tsconvd_LDADD = $(GST_LIBS) $(GIO_LIBS)
```

Рассмотрим подробнее, что должно содержаться в основном файле конфигурации проекта (configure.ac) и зависимостях исходных файлов Makefile.am.

В файле configure.ac сначала описывается продукт (имя проекта, версия, где находятся исходники и заголовки и т.п.). Таким образом, autotool подготавливает шаблон для продукта. Затем проверяется наличие компиляторов и необходимых системных библиотек, а также отдельных функций в этих библиотеках.

На этапе конфигурации будут создаваться небольшие тестовые программы. Они будут тут же компилироваться выбранным компилятором, запускаться, и проверять наличие системных функций. Если эти функции есть, то в config.h будет указано их наличие. В тексте программы можно указать реакцию на наличие или отсутствие тех, или иных системных функций в соответствии с config.h.

Второй файл, Makefile.am, содержит описание исходников, название модуля, и специфические ключи компиляции для каждого каталога сборки.



На слайде приведен процесс конфигурации проекта. После скачивания ПО запускается команда ./configure, и на основании текущей конфигурации создается файл config.h и необходимые платформозависимые файлы. После этого используется команда make, и все собранные файлы при помощи команды make install распределяются в необходимые системные каталоги.

При этом существует большое количество умолчаний, например, о местах хранения файлов, и ./configure выдает именно те директории, где должны храниться конфигурационные файлы.

Все эти умолчания можно изменить при помощи задания опций конфигурации, которые можно получить при помощи ./configure -help

## 51. Сервера сборки/непрерывной интеграции.



### Сервер сборки/ непрерывной интеграции

- Автоматически собирает продукт.
- Новая версия в репозитории в ветке мастер?
  - Скачать, собрать, запустить тесты, подготовить war, создать отчеты.
- Интерфейс для доступа к различным версиям продукта.
- Примеры: CruiseControl, Jenkins, Travis CI...

Указанные выше средства и способы удобны для однократной сборки продукта. Для выполнения этой операции в автоматическом режиме существуют отдельные средства, называемые серверами сборки или серверами непрерывной интеграции. Их основное назначение — сборка новой версии продукта при наступлении заданных администратором или разработчиком условий. Такими условиями могут быть новое обновление исходных кодов в ветке "мастер", наступление определенного момента времени (пример: ночная или еженедельная сборка) и т.д.

Помимо, собственно, сборки, билд-сервер может проводить тесты, снимать метрики с программного кода, производить автозапуск и т.д.

Сервер сборки также предоставляет доступ ко всем существенным собранным версиям продукта для скачивания и / или немедленной установки у пользователя.

Примерами таких систем для Java являются Jenkins и Travis CI.

Непрерывная интеграция (CI, англ. Continuous Integration) — практика разработки программного обеспечения, которая заключается в постоянном слиянии рабочих копий в общую основную ветвь разработки (до нескольких раз в день) и выполнении частых автоматизированных сборок проекта для скорейшего выявления потенциальных дефектов и решения интеграционных проблем. В обычном проекте, где над разными частями системы разработчики трудятся независимо, стадия интеграции является заключительной. Она может непредсказуемо задержать окончание работ.

Переход к непрерывной интеграции позволяет снизить трудоёмкость интеграции и сделать её более предсказуемой за счёт наиболее раннего обнаружения и устранения ошибок и противоречий, но основным преимуществом является сокращение стоимости исправления дефекта, за счёт раннего его выявления.

| Плюсы                                                                                                                               | Минусы                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| проблемы интеграции выявляются и исправляются быстро, что оказывается дешевле                                                       | значительные затраты на поддержку работы непрерывной интеграции                         |
| немедленный прогон модульных тестов для свежих изменений                                                                            | необходимость в дополнительных вычислительных ресурсах под нужды непрерывной интеграции |
| постоянное наличие текущей стабильной версии вместе с продуктами сборки — для тестирования, демонстрации, и т. п.                   |                                                                                         |
| немедленный эффект от неполного или неработающего кода приучает разработчиков к работе в итеративном режиме с более коротким циклом |                                                                                         |

## 52. Основные понятия тестирования. Цели тестирования.

Цели тестирования (Testing goals)

1. Показать разработчику и клиенту, что программное обеспечение отвечает заявленным требованиям.
2. Найти ситуации, когда программное обеспечение ведет себя ошибочно, нежелательно или не соответствует спецификации.
3. Создать предпосылки для предупреждения возникновения ошибок в программном обеспечении.

## 53. Понятие полного тестового покрытия и его достижимости. Пример.

### 1. Полное тестовое покрытие 完整测试覆盖度

Это степень выполнения кода тестами. Основные метрики: покрытие операторов, ветвей, условий, путей, требований. 指测试执行时覆盖代码的程度，常用指标：语句覆盖、分支覆盖、条件覆盖、路径覆盖、需求覆盖

### 2. Достижимость

- Теоретически: для простых программ без циклов 100% покрытие путей достижимо.理论：对无循环的小程序，100%路径覆盖可行。
- Практически: в больших системах невозможно из-за взрыва путей, недостижимого кода и внешних зависимостей. 实际：大型系统几乎不可能，原因包括：路径组合爆炸、不可达代码、外部环境依赖。
- Важно: 100% покрытие не гарантирует отсутствие ошибок. Нужен риск-ориентированный подход. 结论：100%覆盖≠无缺陷，应风险驱动，优先核心模块。

### 3. Пример

Функция классификации возраста (Child/Adult/Senior/Invalid).

Граничные значения (-5, 0, 18, 65) дают 100% покрытие ветвей.

Если добавить цикл, число путей бесконечно → 100% покрытие путей недостижимо.

Даже при полном покрытии ветвей возможны логические или граничные ошибки, поэтому покрытие — лишь один из критериев.

## 54. Статическое и динамическое тестирование.

1. Статическое тестирование – тип тестирования, который предполагает, что программный код во время тестирования не будет выполняться. При этом само тестирование может быть как ручным, так и автоматизированным.

Статическое тестирование начинается на ранних этапах жизненного цикла ПО и является, соответственно, частью процесса верификации. Для этого типа тестирования в некоторых случаях даже не нужен компьютер – например, при проверке требований.

Большинство статических техник могут быть использованы для «тестирования» любых форм документации, включая вычитку кода, инспекцию проектной документации, функциональной спецификации и требований.

Даже статическое тестирование может быть автоматизировано – например, можно использовать автоматические средства проверки синтаксиса программного кода.

2. **Динамическое тестирование** – тип тестирования, который предполагает запуск программного кода. Таким образом, анализируется поведение программы во время ее работы.

Для выполнения динамического тестирования необходимо чтобы тестируемый программный код был написан, скомпилирован и запущен. При этом, может выполняться проверка внешних параметров работы программы: загрузка процессора, использование памяти, время отклика и т.д. – то есть, ее производительность.

Динамическое тестирование является частью процесса валидации программного обеспечения.

## 55. Автоматизация тестов и ручное тестирование.

Одна из целей автоматизации - иметь возможность повторять тестовый сценарий несколько раз.

Важно: Не все тесты следует автоматизировать! В любой компании есть достаточное количество ручных тестировщиков. Основная причина - тесты к сожалению не покрывают весь функционал и найдется такой путь когда надо будет посмотреть быстренько что нибудь в ручную.

- **Регрессионное тестирование** - такое тестирование в котором при исправлении какого либо дефекта в системе, мы должны понять(протестировать) что у нас не развалились другие модули.
- **Повторение тестового сценария** - используется при регрессионном тестировании.
- **Сокращение ручного труда** - тут зависит от ситуации. Иногда ручному тестировщику быстрее сделать тест. А для того чтобы разработать автоматизированный тест тоже нужно время немалое. Поэтому определенно сказать что автоматизированные тесты сокращают количество ручного труда нельзя. В каких то сократит в каких то нет. Но чем больше ПО чем больше нужно сокращать ручной труд и увеличивать количество автоматизированных тестов.
- **Приемочное тестирование** - Автоматизация тестов может использоваться как “чек” к приемочному тестированию.
- **Проверка одного приложения в разных окружениях** - Ну типо есть какая то управлялка тестами и она запускает на разных ос тесты эти. Т е отпадает необходимость по отдельности на каждой платформе проверять тесты. (можно реализовать например с помощью Selenium)

Примеры программ для автоматизированного тестирования: Selenium, TestingWhiz, Sahi, Watir.

## 56. Источники данных для тестирования. Роли и деятельности в тестировании.

Источники данных для тестов:

- Метод черного ящика - описания ПО
  - спецификации, требования, дизайн
  - запуск и сравнение результатов с эталоном
- Метод белого ящика - исходный код
  - переходы, утверждения, условия
  - анализ путей, структуры
- Опыт
- Модели
  - UML

Примечание: подробнее о методе “черного ящика” и “белого ящика” смотрите в вопросе №59

Роли и деятельности в тестировании делятся на несколько больших категорий:

- **Проектирование тестов.** Квалификация спецов проектирующих тесты должна быть высокой и включать в себя знание предметной области, особенностей UI, а также дискретной математики, навыков проги и тестирования
- **Автоматизация тестов.** Обычно этим занимаются прогеры, которые специализируются на разработке тестов скриптов или программ. Принято, что модульные тесты пишут сами разрабы, а интеграционные и системные тесты пишут отдельно выделенные прогеры с глубокими представлениями о архитектуре и взаимодействии различных частей приложения.
- **Непосредственное исполнение тестов** не требует высокой квалификации персонала. Тем не менее, этот персонал должен знать тестовую инфраструктуру, принципы работы с тестируемым приложением и обладать элементарными скилами работы с компом.
- Для **анализа результатов тестов** требуются как знания в предметной области, так и технические знания. Поэтому для данной деятельности необходимо достаточно высокая квалификация персонала.

**Мини вывод** - для организации и проведения полноценного тестирования нужна квалифицированная подготовка, и ко многим участникам процесса тестирования программы предъявляются более высокие требования, чем к ее рядовым разработкам.

## 57. Понятие тестового случая и сценария.

**Тестовый случай** состоит из набора входных значений, предусловий выполнения, ожидаемых результатов и постусловий.

- Входные значения - набор данных, на которых разрабатываемое ПО должно вести себя определенным образом с точки зрения спецификации требований к ПО или других источников инфы о поведении программы.
- Ожидаемые результаты - создаются как часть спецификаций тестовых сценариев и включает в себя выходные данные, изменения в данных и состояниях, и любые иные последствия теста.

Примечание: В идеальных условиях ожидаемые результаты должны быть определены до момента выполнения теста (TDD - test-driven development). TDD - разработка через тестирование. Смысл - разраб сначала описывает тестовое покрытие и разрабатывает тесты для этого покрытия. Затем он начинает разрабатывать ПО, и с каждым его запуском растет кол-во тестов выполненных успешно. Разработка считается завершенной, когда будут успешно выполнены все тесты.

- Тестовый случай должен быть повторяемый, то есть одинаковый набор входных значений и состояний должен приводить к одинаковым выходным значениям или состояниям
- Для повторяемости теста желательно автоматизировать его поведение.
- Необходимо тестить как нормальные пользовательские сценарии так и те, которые приводят к ошибкам.

**Тестовый сценарий** - это последовательность тестовых случаев!

В тестовых сценариях должно быть предусмотрено появление как положительной, так и отрицательной реакции системы на действия пользователей. Для неправильных действий пользователя также создаются тестовые сценарии.

## 58. Выбор тестового покрытия и количества тестов. Анализ эквивалентности.

При планировании тестированию следует соблюдать баланс между качеством и скоростью вывода продукта в эксплуатацию. Больше тестов -> покрытие растет -> рост качества, но также растет затрачиваемое время и ресурсы.

Количество тестовых сценариев необходимо выбирать с учетом того, что полное тестовое покрытие недостижимо.

Рассмотрим набор методов, которые позволяют определить достаточное тестовое покрытие:

- **Эквивалентное разбиение (партиции эквивалентности)** - анализ граничных значений, внутри которых тестируемая функция ведет себя одинакового.
- **Таблица решений (альтернатив)** - Составляется таблица, отражающая комбинации входных данных с соответствующими выходными данными, которая может быть использована для проектирования тестовых сценариев, которые необходимо провести для группы таких сочетаний.
- **Таблицы переходов** - выделяются явные состояния внутри системы, определяются переходы между этими состояниями, которые далее покрываются тестами
- **Сценарии использования** - могут служить источником данных для формирования тестового покрытия. При этом в каждый описанный сценарий добавляются конкретные значения, вводимые пользователем. Нужно учитывать как основные, так и альтернативные пути сценария. Обычно каждому такому пользовательскому сценарию соответствует целая группа тестовых сценариев.

**Анализ эквивалентности** - функция или модуль разбиваются на участки, где программа ведет себя одинакового (эквивалентно). Внутри каждого участка формируется свой набор тестовых случаев. Если таких участков относительно немного, это позволяет резко сократить кол-во тестовых случаев. Отдельные тесты составляются для граничных значений участков.

## 59. Модульное тестирование. Junit 4.

[Почитать](#)

### Модульное тестирование

**Модуль** - это компонент, который необходимо протестировать отдельно от остального программного продукта. Модуль выполняет некую законченную функцию.

- IEEE 610
- Модули описаны в дизайне
- Для тестирования необходимо изолировать модуль из системы

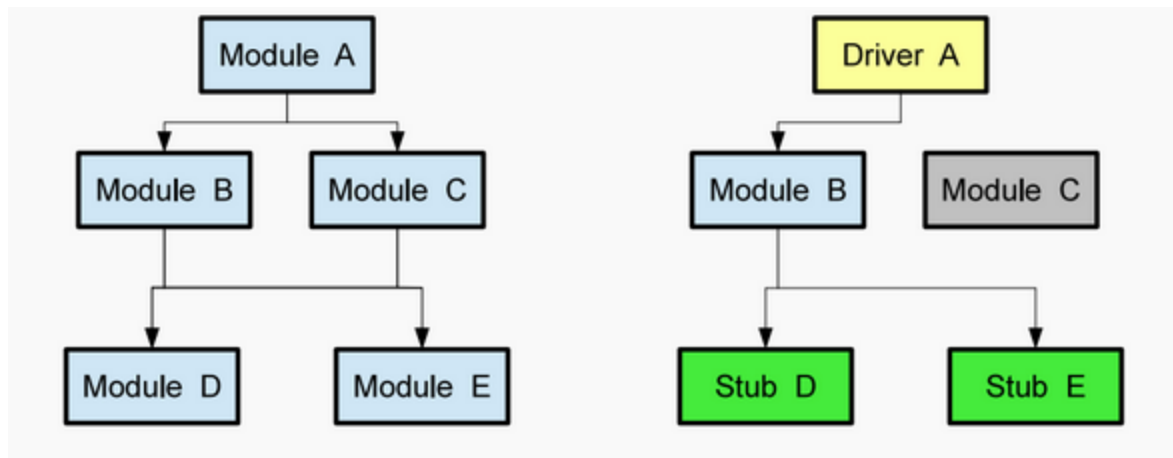
Для проведения модульного тестирования, модуль необходимо изолировать из системы. Для этого юзается драйвера (вместо вызывающего модуля) и заглушки (вместо подчиненного модуля).

Драйвер - компонент, вызывающий модули и обеспечивающий последовательность тестирования. Драйвер должен вызывать тестируемый модуль с различными входными

параметрами и условиями.

Заглушка - ведет себя подобно подчиненному модулю, имеет тот же интерфейс, но гораздо более простую реализацию. При вызове заглушка возвращает заранее определенные значения.

Картинка для понимания:



Стратегии модульного тестирования:

- метод “Черного ящика” - подразумевается, что внутреннее содержимое проги скрыто. Работа при таком тестировании идет на уровне спецификации или на основе опыта составления тестов. Тесты подают на вход проги исходные данные и сравнивают результат с известным эталоном. Источниками данных для тестов являются спецификации, требования и дизайн.
- метод “Белого ящика” - У нас есть возможность исследовать исходники. Плюс данный метод дает возможность оценить размер тестовго покрытия.

Вкратце:

- Черный ящик - не знаем что находится под капотом
- Белый ящик - знаем что находится под капотом

## JUnit

- простейший фреймворк, позволяющий создать модульные тесты и выполнить их в определенном окружении. JUnit4 построен на аннотациях. Как работает - фреймворк последовательно просматривает загружаемые классы (с помощью reflection API) и ищет в них нужную аннотацию. Все методы с найденной аннотацией запускаются (мб и в разных потоках)

- JUnit фреймворк обеспечивает:
  - Аннотации для маркировки метода как теста `@Test`.
  - Аннотации для маркировки действий до и после теста `@Before`, `@After`, `@BeforeClass`, `@AfterClass`.
  - Методы для проверки (assertion).
  - UI, журнал тестов...

Внутри тестового метода проверяется тестовое покрытие на соответствие определенным условиям при помощи специальных функций проверки, которые называются assertion.

В JUnit4 существует большой набор таких функций, которые могут:

- проверять на равенство
- проверять на совпадения
- появления исключительной ситуации и тд

Результаты тестов заносятся в специальный журнал для последующего анализа.

Примечание: Последовательность тестов в JUnit не регламентирована (порядок конечно можно и искусственно определить, но по дефолту считается, что все тесты выполняются параллельно и независимо). JUnit сам определяет, каким образом будет запущено тестирование.

## 60. Интеграционное тестирование. Стратегии интеграции.

Интеграционное тестирование - это тип тестирования, при котором программные модули объединяются логически и тестируются как группа. Как правило, программный продукт состоит из нескольких программных модулей, написанных разными программистами. Целью нашего тестирования является выявление багов при взаимодействии между этими программными модулями и в первую очередь направлен на проверку обмена данными между этими самими модулями. ([https://logrocon.ru/news/intgration\\_testing](https://logrocon.ru/news/intgration_testing))

Программная инженерия задает различные стратегии интеграционного тестирования:

- Подход Большого взрыва.

- Инкрементальный подход:
  - Нисходящий подход (сверху вниз)
  - Подход «снизу вверх»
  - Сэндвич - комбинация «сверху вниз» и «снизу вверх»

## **61. Функциональное и End-to-End тестирование. Selenium.тестирование. Selenium.**

Selenium-Это инструмент для автоматизации действий веб-браузера. В большинстве случаев используется для тестирования Web-приложений, но этим не ограничивается. В частности, он может быть использован для решения рутинных задач администрирования сайта или регулярного получения данных из различных источников.

Функциональное тестирование — это тестирование ПО в целях проверки реализуемости функциональных требований, то есть способности ПО в определённых условиях решать задачи, нужные пользователям.

## **62. Техники статического тестирования. Статический анализ кода.**

Статический анализ кода — анализ программного обеспечения, производимый (в отличие от динамического анализа) без реального выполнения исследуемых программ. В большинстве случаев анализ производится над какой-либо версией исходного кода, хотя иногда анализу подвергается какой-нибудь вид объектного кода, например Р-код или код на MSIL. Термин обычно применяют к анализу, производимому специальным программным обеспечением, тогда как ручной анализ называют «program understanding», «program comprehension» (пониманием или постижением программы).

В зависимости от используемого инструмента глубина анализа может варьироваться от определения поведения отдельных операторов до анализа, включающего весь имеющийся исходный код. Способы использования полученной в ходе анализа информации также различны — от выявления мест, возможно содержащих ошибки (утилиты типа Lint), до формальных методов, позволяющих математически доказать какие-либо свойства программы (например, соответствие поведения спецификации).

## 63. Тестирование системы в целом. Системное тестирование. Тестирование производительности.

### 1. Тестирование системы в целом (Общее системное тестирование)

Тестирование системы в целом — это процесс проверки всей программной системы (включая аппаратное обеспечение, сеть, базы данных, внешние интерфейсы и т. д.) как единого целого в реальных или симулированных условиях эксплуатации. Основная цель — убедиться, что поведение всей системы полностью соответствует бизнес-требованиям и проектным спецификациям. 系统整体测试是指在真实或模拟的完整运行环境下，将整个软件系统（包括硬件、网络、数据库、外部接口等）作为一个整体进行验证的过程。它的核心目标是确保系统整体行为符合业务需求和设计规格

### 2. Системное тестирование (System Testing)

- **Определение:** Системное тестирование — это ключевой уровень тестирования в жизненном цикле ПО, выполняемый после интеграционного тестирования и перед приёмочным тестированием. Оно направлено на всестороннюю проверку функциональных и нефункциональных требований. 系统测试是软件测试生命周期中一个关键级别，发生在集成测试之后、验收测试之前。它主要针对功能性和非功能性需求进行全面验证。
- **Среда выполнения:** Обязательно проводится в отдельной тестовой среде, максимально приближенной к продуктивной. 必须在尽量接近生产环境的独立测试环境中执行。
- **Основные области внимания:**
  - **Функциональность:** Проверка корректности работы всех модулей согласно спецификации требований. 功能性：验证所有功能模块是否按需求规格说明书正确工作。
  - **Нефункциональные аспекты:** Включают производительность, безопасность, удобство использования, совместимость, надёжность и др. 非功能性：包括性能、安全性、可用性、兼容性、可靠性等。
- **Основные методы:** Преимущественно используется чёрный ящик (тестирование по входным и выходным данным без анализа внутреннего кода). Применяются методы эквивалентного разделения, анализа граничных значений, тестирование на основе сценариев использования и т.д. 黑盒测试为主（不关注内部代码逻辑，只关注输入和输出），常用技术包括等价类划分、边界值分析、用例测试等。

### 3. Тестирование производительности (Performance Testing)

Тестирование производительности — это наиболее важная и сложная часть нефункционального системного тестирования. Оно оценивает скорость, время отклика, стабильность, использование ресурсов и масштабируемость системы под определённой рабочей нагрузкой. Обычно делится на следующие подвиды: 性能测试是非功能性系统测试中

最核心、最复杂的一类，用于评估系统在特定工作负载下的速度、响应时间、稳定性、资源利用率和可扩展性。它通常分为以下几个子类型：

- Нагрузочное тестирование (Load Testing) — моделирование ожидаемого количества одновременных пользователей для проверки достижения целевых показателей производительности. 负载测试
- Стресс-тестирование (Stress Testing) — постепенное увеличение нагрузки до точки отказа системы, чтобы определить её предельную пропускную способность и способность восстанавливаться после сбоев. 压力测试
- Тестирование стабильности / длительной нагрузки (Soak / Endurance Testing) — постоянная подача средней нагрузки в течение длительного времени (часы или сутки) для выявления утечек памяти, истощения ресурсов и других долгосрочных проблем. 耐久性/浸泡测试
- Тестирование пиковых нагрузок (Spike Testing) — моделирование резких, внезапных скачков трафика для проверки способности системы справляться с кратковременными всплесками запросов. 峰值测试
- Тестирование масштабируемости (Scalability Testing) — постепенное увеличение аппаратных ресурсов или количества пользователей для проверки линейности роста производительности системы. 可扩展性测试

## 64. Тестирование системы в целом. Альфа- и бета-тестирование.

### 1. Тестирование системы в целом

Определение: Это сквозное (E2E) тестирование всей системы (софт + железо + сеть + внешние интерфейсы) в среде, максимально приближенной к боевой.

Цель: Проверка не только функций, но и взаимодействия компонентов, общей производительности, безопасности и надёжности.

### 2. Альфа-тестирование (α-тестирование)

Кто проводит: Внутренняя команда (разработчики, тестировщики, продакт-менеджеры).

Где: В среде разработки или внутренней тестовой лаборатории (контролируемая среда).

Когда: Перед бета-тестированием.

Особенности: Имитирует действия пользователей, цель — быстро выявить грубые дефекты и логические ошибки.

### 3. Бета-тестирование (β-тестирование)

Кто проводит: Реальные конечные пользователи (внешние тестировщики или фокус-группа).

Где: В реальной боевой или пред релизной среде (неконтролируемая).

Когда: После альфа-тестирования, перед официальным релизом.

Особенности: Основной фокус — пользовательский опыт (UX), совместимость (разные устройства/браузеры) и сложные сценарии, которые не выявили внутренние тесты.

#### 1. 整体系统测试（Тестирование системы в целом）

定义：在尽可能接近生产环境的完整环境中，对整个系统（软件+硬件+网络+外部接口）进行端到端（E2E）验证。

目标：不仅检查功能，还检查组件之间的交互、整体性能、安全性和可靠性。

#### 2. Alpha测试（Альфа-тестирование）

执行方：内部团队（开发人员、测试工程师、产品经理）。

环境：开发环境或内部测试实验室（受控环境）。

时间：在Beta测试之前进行。

特点：模拟内部用户操作，目的是快速发现严重的缺陷和逻辑错误。

#### 3. Beta测试（Бета-тестирование）

执行方：真实的最终用户（外部用户或种子用户）。

环境：真实的生产环境或预发布环境（非受控）。

时间：在Alpha测试之后、正式发布之前进行。

特点：重点验证用户体验（UX）、兼容性（不同设备/浏览器）以及内部测试未覆盖的复杂真实场景。

## 65. Аспекты быстродействия системы. Влияние средств измерения на результаты.

- выбрать критерии оценки
- выбрать средства измерения
  - Неинтрузивные средства измерения = не влияют на результат
- выбрать нагрузку
  - стараться имитировать нагрузку, похожую на настоящую
  - это бывает сложная задача
- анализ результатов
- сделать небольшое изменение в системе, один участок кода
  - чтобы протестировать именно то, что изменяли
- повторять до полного удовлетворения

## 66. Ключевые характеристики производительности.

стр. 150

- Время отклика,
  - ожидание от воздействия на систему до получения первых данных
- Время полного обслуживания
  - от начала запроса до полного его завершения
- Пропускная способность
  - сколько запросов в единицу времени можно сделать
- Утилизация и ожидание ресурса
- точка насыщения и масштабируемость
- Эффективность
  - 100% загруженность CPU не означает, что наша программа работает эффективно
- Ускорение и прирост производительности
  - что произошло после тестирования и улучшения производительности системы
- Java: Скорость запуска memory footprint
  - связано с Garbage collector
  - то, как программа себя ведет внутри памяти
  - сервера приложений могут запускаться очень долго

## 67. Нисходящий метод поиска узких мест.

стр. 151

Классический "нисходящий" метод поиска узких мест:

- Исключение ошибок аппаратуры и администратора:
  - системные журналы, файлы конфигурации системы и прикладного ПО
- Общесистемный и межузловой мониторинг:
  - ОС сама собирает большое количество информации
  - Программисты обычно используют сторонние утилиты
  - Если у вас кластер, для них есть отдельный набор своих утилит по мониторингу
  - Анализируются: CPU, память, подсистема I/O, дисковые массивы, сеть, каналы передачи, ОС, виртуализация
- Мониторинг приложения:
  - алгоритмы, проблемные API, многопоток, блокировки, синхронизации
  - сторонние прикладные API могут вредить производительности системы. Нужно четко понимать зачем нужна та или иная библиотека и какие накладные расходы она может дать. Можно она избыточная и проще будет написать какие-то модули руками
- Мониторинг микроархитектуры:
  - выравнивание данных — уверенность, что данные попадают в нужную строчку кэша
  - оптимизация кэшей
  - пузырьки конвейера
  - предсказание переходов

## **68. Пирамида памяти и ее влияние на производительность.**

|                 | Объем           | Тд        | *     | Тип       | Управл.                    |
|-----------------|-----------------|-----------|-------|-----------|----------------------------|
| CPU             | 100-1000 б.     | <1нс      | 1с    | Регистр   | компилятор                 |
| L1 Cache        | 32-128Кб        | 1-4нс     | 4с    | Ассоц.    | аппаратура                 |
| L2-L3 Cache     | 0.5-32Мб        | 8-20нс    | 30с   | Ассоц.    | аппаратура                 |
| Основная память | 0.5Гб-4Тб       | 60-200нс  | 3-10м | Адресная  | malloc, free<br>slab-alloc |
| SSD             | 128Гб-1Тб/drive | 25-250мкс | 5д    | Блочн.    | open<br>read/write         |
| Жесткие диски   | 0.5Тб-4Тб/drive | 5-20мс    | 4мес  | Блочн.    | open<br>read/write         |
| Магнитные ленты | 1-6Тб/к         | 1-240с    | 200л  | Последов. | программно                 |

**NB: при 2% cache miss производительность падает ниже 60% от идеальной!**

- CPU : самая быстрая и самая дорогая память
- L1 : очень быстрая память. Находится максимально близко к ядрам процессора
- L2, L3 : тоже находятся внутри процессора.
- RAM : бОльшее время доступа, но и внушительный объем.
- SSD : Существенно медленнее оперативной памяти. Условно в 1000 раз медленнее
- HDD : Еще примерно в 1000 раз медленнее
  - масштаб времени относительно процессора

Обычно, чем выше в пирамиде памяти находится тип памяти, используемой системы, тем быстрее выполняется код. Недостаток верхних уровней(быстрой памяти) в том, что обычно они очень маленького размера. Поэтому приходится идти на компромиссы и искать оптимальную память по соотношению скорости/объема. Примером работы с памятью является виртуальная память, которая из-за нехватки оперативной памяти использует постоянную.

## 69. Мониторинг производительности: процессы.

стр. 153

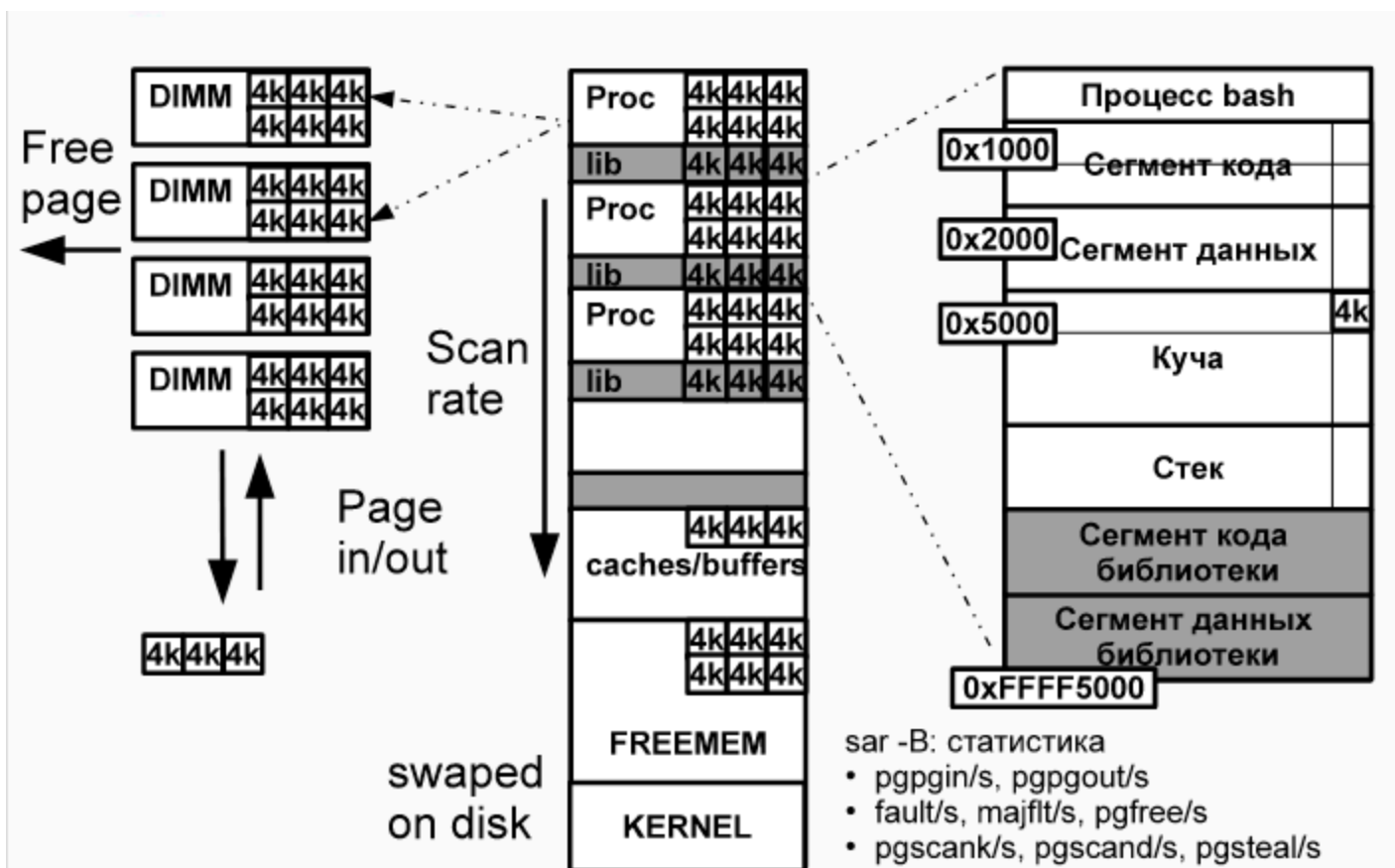
- CPU : user%, system%, idle% (простой), прерывания, смена контекста (ctxt\_sw), средняя нагрузка

- IO : число байт читаемых и записываемых за секунду, и за какое количество операций это происходит; время ожидания
- VM : (Виртуальная память) : скорость выделения, освобождения страниц, пейджинга, swar'инга
- Сеть : счетчики принятых/переданных данных, коллизий и тд

Системный мониторинг неинтрузивен (не оказывает влияния на производительность, так как счетчики автоматически включаются ОС).

## 70. Мониторинг производительности: виртуальная память.

Виртуальная память — метод управления памятью компьютера, позволяющий выполнять программы, требующие больше оперативной памяти, чем имеется в компьютере, путём автоматического перемещения частей программы между основной памятью и вторичным хранилищем (например, жёстким диском).



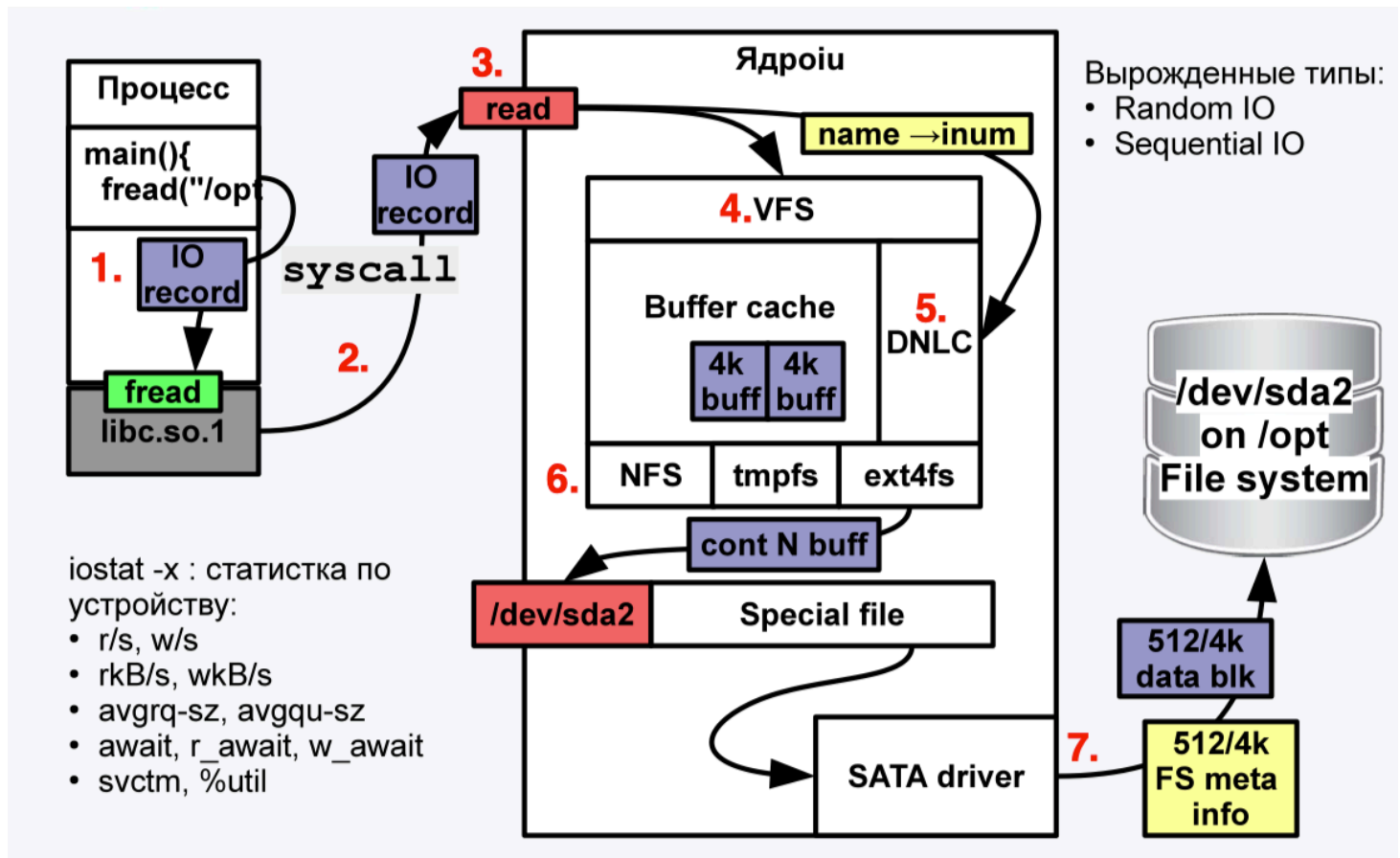
**Виртуальная память** — способ защиты системы. Внутри VM приложение думает, что работает в одних и тех же адресах. Потом просто эта память маппится в реальную.

Можно использовать странички повторно, использовать

- **paging** : сброс неиспользуемых страниц на диск или их уничтожение,
- **совместное использование библиотек** : VM позволяет сделать так, чтобы одна страничка памяти была подключена к разным процессам, вместо создания копии для каждого
- **swapping** : процесс целиком уходит в область подкачки

## 71. Мониторинг производительности: буферизированный файловый ввод-вывод.

стр. 156



1. IO record - сколько байт нужно прочитать
2. Запрос к ядру (ссылка на запись с IO передается вместе с синхронным программным прерыванием)
3. Вызов функции ядра **read** (точка доступа внутри ядра)
4. IO в ядре попадает в VFS (Virtual File System)
5. Имя файла преобразуется в DNLC - Directory Name Lookup Cache - кэш, ускоряющий обработку имен файлов
  - если директория содержит большое количество имен файлов, кэша не хватает —> система начинает тупить <—> ОС берет информацию с диска, а не с быстрой памяти

- для экономии ресурсов ОС будет пользоваться в первую очередь буферным кэшем

#### 6. Реализации модулей ядра физических файловых систем

- Tmpfs — временное файловое хранилище во многих Unix-подобных ОС.

Предназначена для монтирования файловой системы, но размещается в ОЗУ вместо физического диска.

#### 7. К точке монтирования подключается устройство, далее работает драйвер

Полезная утилита `iostat` . Позволяет наблюдать:

- `r/s`, `w/s` - количество чтений и записей
- `rkB/s`, `wkB/s` - объем `r/w` данных
- `avgrq-sz` - среднее время запросов
- `await`, `r_await`, `w_await` - время ожидания
- `svctm` - время обслуживания
- `%util` - загруженность диска

Процент занятости устройства - `%used`

Случайный (random IO) и последовательный (sequential IO) доступ. Первый - чтение из разных частей памяти накопителя. В случае с HDD из-за движения головки и диска теряет время на постановку в нужную точку, в отличие от SSD(движущиеся части отсутствуют).

Последовательный доступ - чтения без трат на перестановку, за счёт чего скорость сводится лишь к пропускной способности чтения диска и передачи.

Узнать характеристику текущего обмена - `iostat`.

## 72. Мониторинг производительности: Windows и Linux.

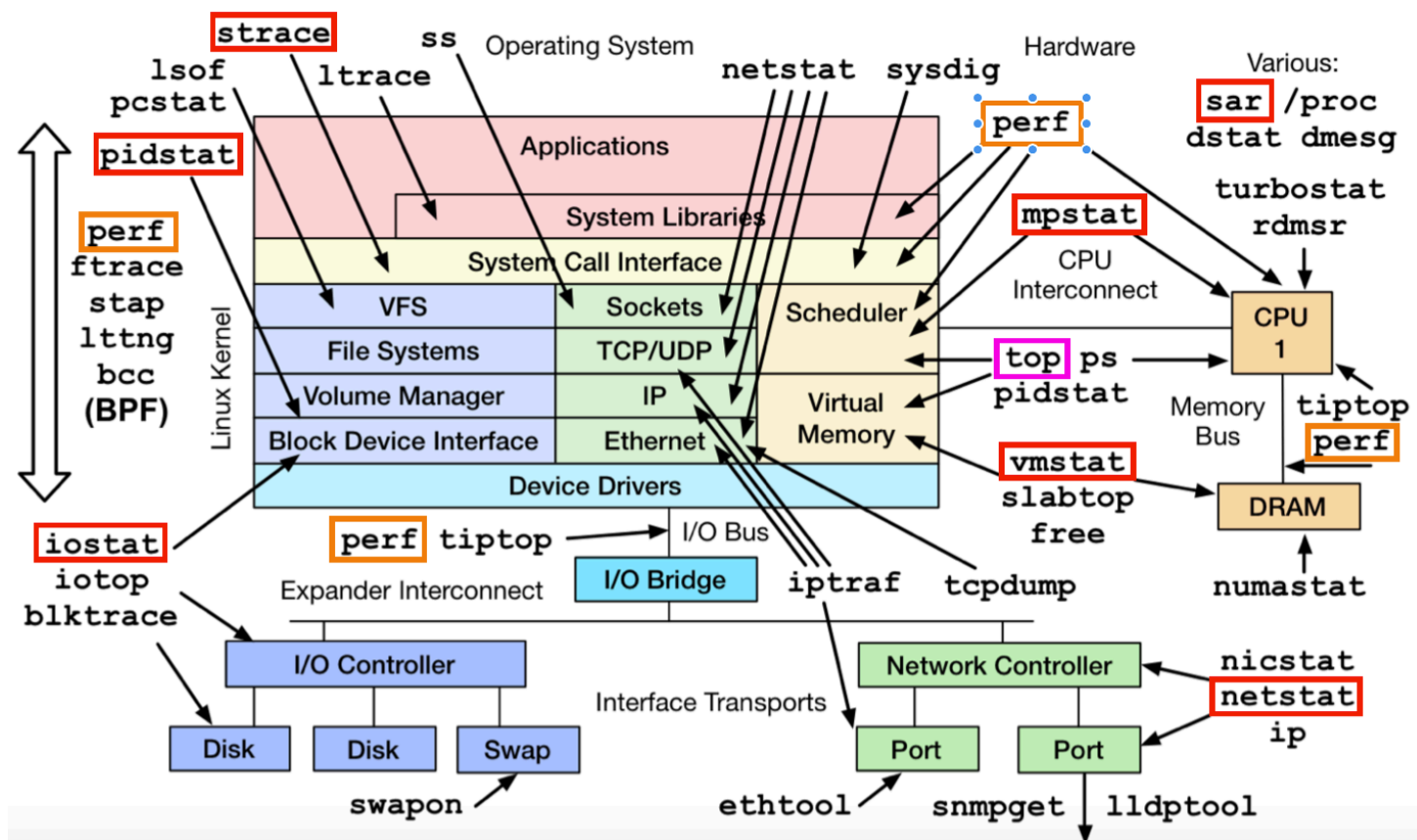
Windows

Task Manager - стандартное, встроенное решение

Resource Monitor & Performance Monitor

Reliability Monitor

Microsoft SysInternals - наиболее точный инструмент, созданный независимо, но в последствии купленный M-soft



В лекции

См конспект

## 73. Системный анализ Linux "за 60 секунд".

- uptime - средняя нагрузка процессорной подсистемы (load average) в кол-ве готовых к выполнению процессов. За 1, 5 и 15 минут для получения динамики за длительный промежуток времени
- dmesg | tail - последние ошибки
- vmstat 1 - проверка виртуальной памяти
- mpstat -P ALL 1 - распределение процессов по CPU
- pidstat 1 - проверка наиболее "горячих" процессов
- iostat -xz 1 - характеристики ввода-вывода
- free -m - проверка исчерпания кэшей/буфферов
- sar -n DEV1 - сетевая статистика по интерфейсам
- sar -n TCP,ETCP 1 - сетевая статистика по соединениям
- top - онлайн-мониторинг параметров - "швейцарский нож" для администратора

## 74. Создание тестовой нагрузки и нагрузчики.

В корпоративных системах наблюдение реальных, критически важных для бизнеса систем часто запрещено из-за страха перед внесением искажений в нормальную работу системы средствами мониторинга, или перед наличием дефектов в средствах. В таких случаях обычно создают отдельную тестовую систему, являющуюся полной копией реальной, и проводят измерения на ней.

Для тестовой системы нужно иметь возможность создавать нагрузку, близкую по характеристикам к реальной пользовательской нагрузке. В тестовой системе также возможно использование интрузивных средств мониторинга.

Нагрузку, аналогичную реальной, можно создать с помощью средства создания синтетической нагрузки или средства записи реальной нагрузки, позволяющего запомнить нагрузку на реальном устройстве и использовать эти данные для тестовой системы. При этом синтетическая нагрузка всегда будет отличаться от реальной, и в средствах создания такой нагрузки используется большое число параметров, позволяющих гибко её настроить.

## 75. Профилирование приложений. Основные подходы.

- Неинтрузивных профилировщиков **не бывает**

Профилировщики нужны для анализа:

- Времени исполнения методов
- Создания объектов в памяти
- Поток / блокировок

Два подхода:

- Внедрение в код напрямую
- Прерывания и дампы памяти с последующими сортировкой и анализом
  - получаем стек вызовов с определенным периодом
  - понимаем сколько примерно занимает исполнение разных методов
  - погрешность в рамках периода есть (если период больше времени отработки функции, ее можно не заметить)

## 76. Компромиссы (trade-offs) в производительности.

- Учите алгоритмы и структуры данных

## 77. Рецепты повышения производительности при высоком %SYS.



### Рецепты: высокий %SYS

- Высокий ввод-вывод (диск, сеть, ...):
  - Реже писать/читать. Сжимать данные.
  - Буферизация, более быстрые устройства.
  - Размер блока передачи.
- Планировщик — большая runqueue / CS:
  - Обратить внимание на потоки (ПО или OS).
- Paging/swapping:
  - Больше памяти системе, меньше процессам.
  - Запретить выгрузку из памяти.
- Другие системные функции ядра:
  - Найти и попытаться исключить.
  - Настраивать ядро.

## 78. Рецепты повышения производительности при высоком %IO wait.



ИТМО ВТ

### Рецепты: высокий %IO wait

- Проблемы приложений:
  - Оптимизировать запросы к диску.
  - Согласовать блок приложения с ОС и дисками.
- Буфера/кэши:
  - Расширить память.
  - Настроить буферы/кэши.
- Аппаратура:
  - Купить новую дисковую подсистему.
  - Купить SSD или flash-кэш (для случайного доступа).

## 79. Рецепты повышения производительности при высоком %Idle.



### Рецепты: высокий %Idle

- Проблемы приложений — мало RUNNABLE:
  - Параллелить всё, что можно.
  - Добавить потоки в пулы приложений.
  - Анализ и оптимизация блокировок.
  - Lock-free алгоритмы.
- Проблемы ОС:
  - Блокировки — можно мониторить через средства ОС.
  - Настройка параметров подсистем ядра.
  - Поиск дефектов в багтрекере.

## 80. Рецепты повышения производительности при высоком %User.



ИТМО ВТ

### Рецепты: высокий %user

- Проблемы приложений:
  - Алгоритмы с меньшей алгоритмической сложностью.
  - Повторное использование объектов.
  - Избавиться от активных опросов и ожиданий в циклах.
  - Искать промахи мимо кэшей и TLB.
  - Учитывать размер и организацию кэшей:
    - Размер кэшей ограничен.
    - Паддинг и разрывание объектов.
    - Использовать более слабые примитивы синхронизации.
  - "Биндить" процессы/потoki к процессорам.
  - Оптимизировать на уровне ассемблера/компилятора.
  - Повысить частоту процессора, увеличить количество ядер, использовать GPU/криптопроцессоры.